Q1. Automation testing: need/importance and criteria/guidelines for selecting automation testing tools.

ANS

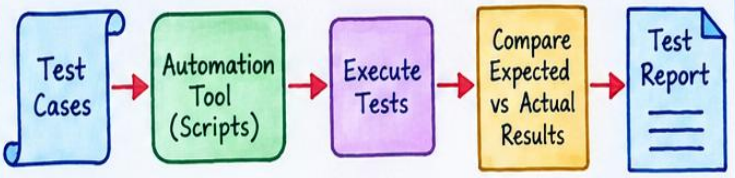
# Automation Testing : Need/Importance and Criteria for Selecting Automation Testing Tools

(10 Marks)

## 1) What is Automation Testing ?

Automation testing is a software testing technique in which test cases are executed automatically using specialized tools or scripts instead of manual execution. It compares the actual results with the expected results and reports the defects.

### Automation Testing Process




## 2) Need / Importance of Automation Testing

1. **High Reusability** - Automated test scripts can be reused multiple times for different builds and releases.
2. **Saves Time** - Executes test cases much faster than manual testing.
3. **Cost Effective (Long Term)** - Reduces manual effort and cost for repetitive testing in the long run.
4. **Improves Test Coverage** - Large number of test cases can be executed including data combinations that are difficult manually.
5. **Higher Accuracy** - Eliminates human errors and gives consistent results.
6. **Supports Regression Testing** - Useful for repeated testing after changes in the application.
7. **Enables Continuous Testing** - Integrates with CI/CD pipeline for frequent builds and faster feedback.
8. **Better Reporting and Logs** - Provides detailed execution reports, logs and helps in defect analysis.
9. **Supports Complex and Repetitive Tests** - Ideal for large, complex and data-driven test scenarios.
10. **Improves Overall Product Quality** - Helps in early defect detection and improves reliability of the software.

### When is Automation Testing Needed ?

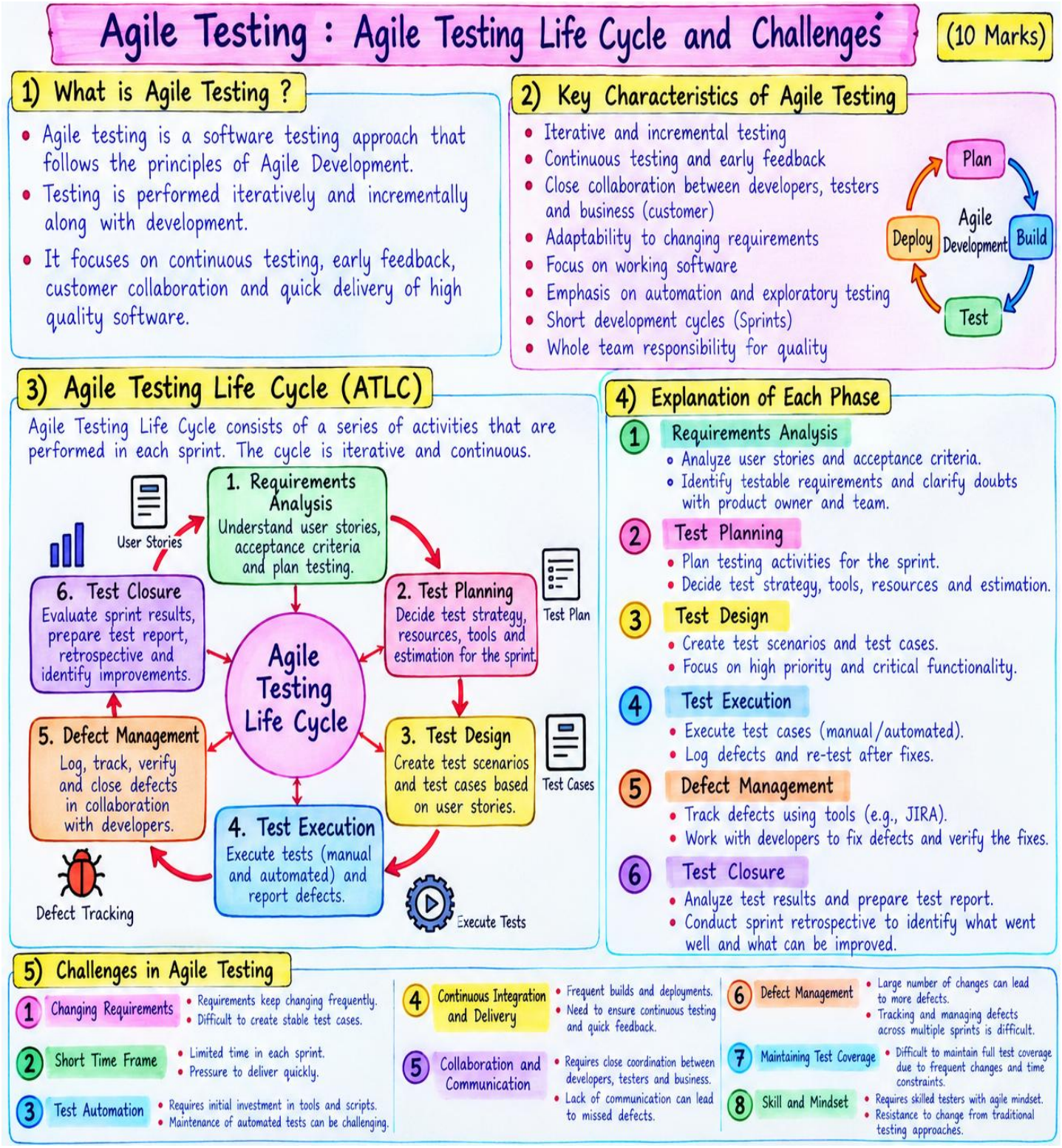
- Large and complex applications
- Frequent releases and changes
- Repetitive test cases
- Critical and time-consuming tests
- Cross-browser / cross-platform testing
- When high test coverage is required

## 3) Criteria / Guidelines for Selecting Automation Testing Tools

1. **Ease of Use** - Simple and easy to learn (low learning curve).
2. **Tool Support and Community** - Good community support, documentation and regular updates.
3. **Technology Compatibility** - Should support the application's technology (web, mobile, API, etc.).
4. **Scripting and Programming Support** - Support for preferred programming languages (Java, Python, etc.).
5. **Features and Capabilities** - Support for required testing types (functional, regression, performance, etc.).
6. **Cost and Licensing** - Open source or affordable licensing cost.
7. **Integration** - Should integrate with CI/CD tools, defect tracking tools (like JIRA) and other ALM tools.
8. **Vendor Support** - Reliable vendor support and regular updates.
9. **Tool Stability and Performance** - Should be stable, reliable and handle large test suites.
10. **Flexibility and Scalability** - Should be flexible for future needs and scalable for large projects.

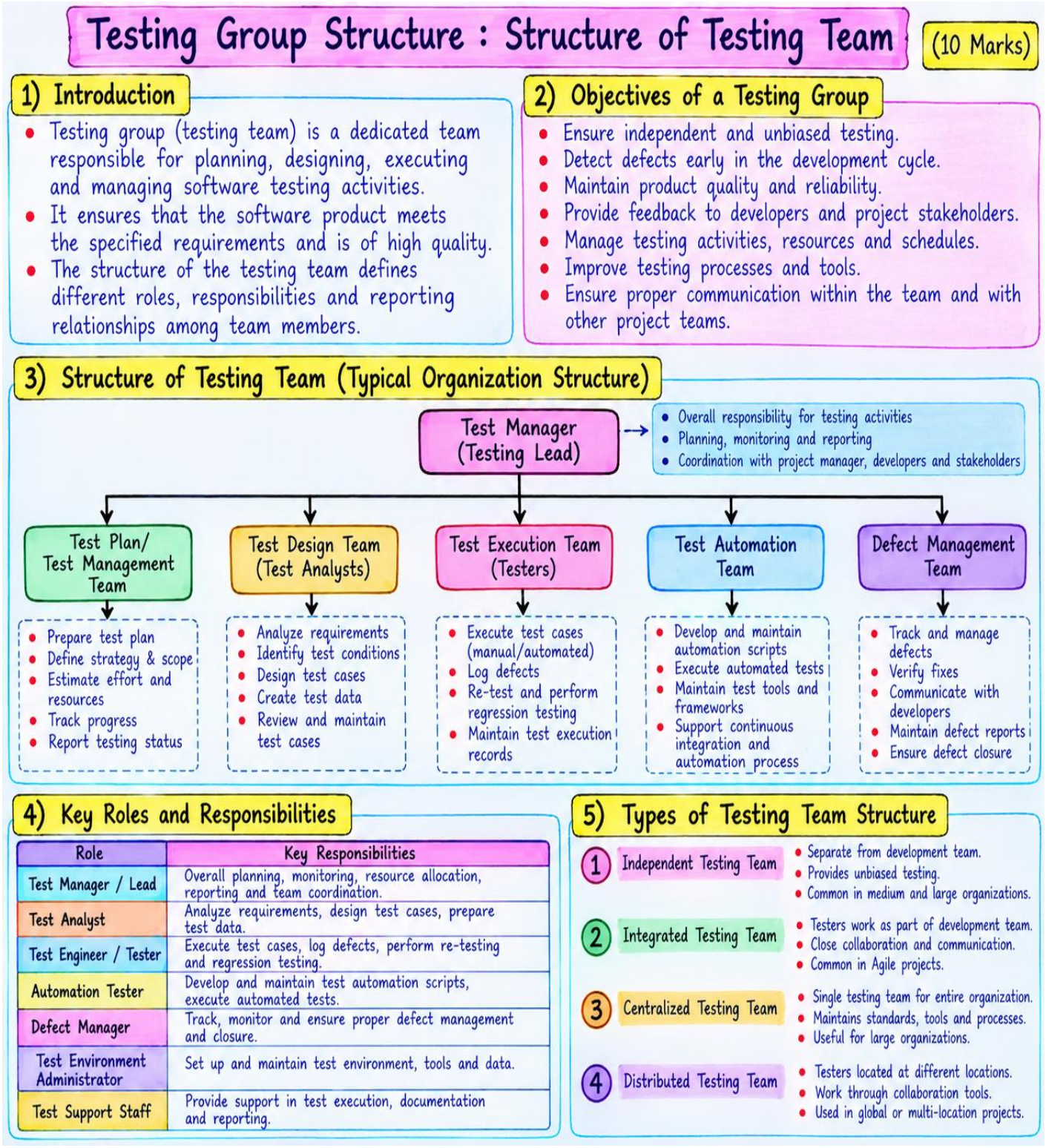
Q2. Agile testing: Agile Testing Life Cycle and challenges / Agile testing and challenges.

ANS



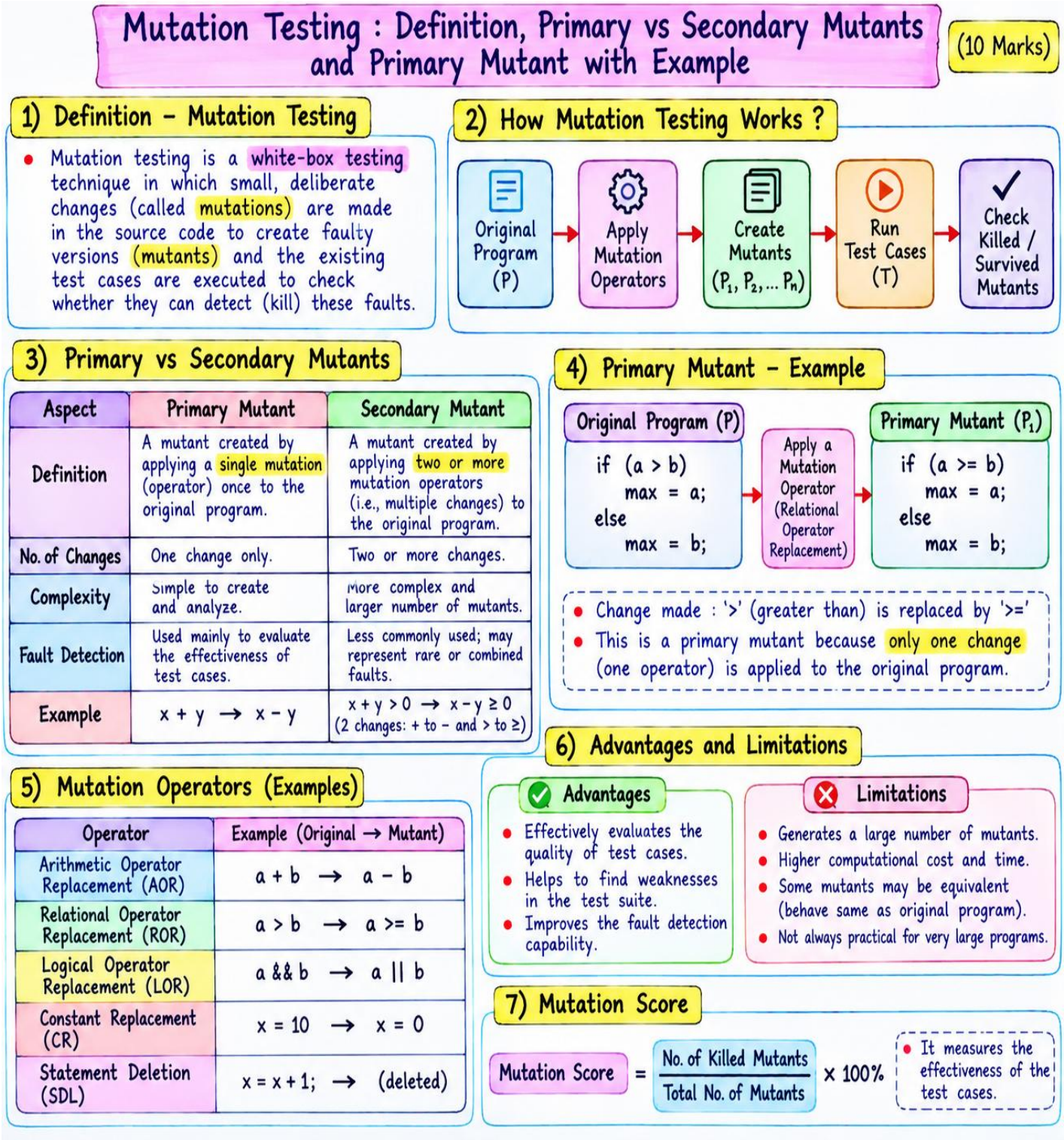
Q3. Testing group structure: explain structure of testing group / testing team.

ANS



Q4. Mutation testing: definition, primary vs secondary mutants, primary mutant with example.

ANS



Q5. Cyclomatic complexity / graph metrics: DD graph, cyclomatic complexity and related graph-metric calculations.

ANS

# Cyclomatic Complexity / Graph Metrics : DD Graph, Cyclomatic Complexity and Related Graph-Metric Calculations

(10 Marks)

## 1) Cyclomatic Complexity – Definition

- Cyclomatic Complexity (CC) is a software metric proposed by McCabe to measure the complexity of a program's control flow.
- It gives the number of linearly independent paths in the program.
- Higher CC indicates more decision points, more test cases and higher testing effort.

## 2) Formulas for Cyclomatic Complexity

$CC = E - N + 2P$

Where,
- E = Number of edges (links)
- N = Number of nodes (vertices)
- P = Number of connected components (usually P = 1 for a single program)

$CC = D + 1$

Where, D = Number of decision (predicate) nodes (e.g., if, while, for, case, etc.)

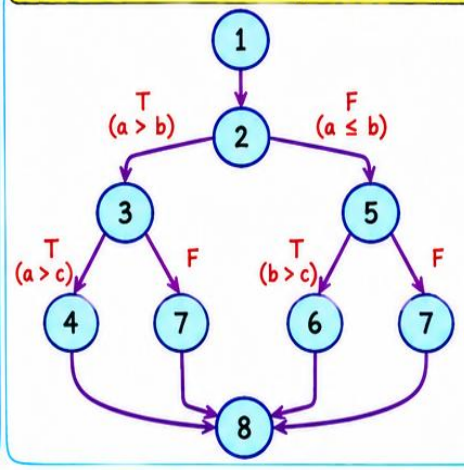
CC = Number of Regions (R)

Where, R = Number of regions in the flow graph (including the outer region)

## 3) Example Program

```
int max(int a, int b, int c)
{
    if (a > b)
    {
        if (a > c)
            max = a;
        else
            max = c;
    }
    else
    {
        if (b > c)
            max = b;
        else
            max = c;
    }
    return max;
}
```

## 4) Decision / Flow Graph (DD Graph)




## 5) Graph Metrics (for the DD Graph)

| Metric | Count / Value |
|---|---|
| Number of Nodes (N) | 8 |
| Number of Edges (E) | 9 |
| Number of Decision Nodes (D) | 3 (nodes 2, 3, 5) |
| Number of Regions (R) | 4 |
| Connected Components (P) | 1 |

## 6) Cyclomatic Complexity Calculations

**(ii) Edge-Node Method**

$CC = E - N + 2P$
$= 9 - 8 + 2(1)$
$= 3$

- Using edges, nodes and connected components.

**(ii) Decision Node Method**

$CC = D + 1$
$= 3 + 1$
$= 4$

- D = 3 (nodes 2, 3, 5) which have a decision (if) condition.

**(iii) Region Method**

$CC = R$
$= 4$

- Number of regions in the flow graph = 4.

## 7) Results & Interpretation

- Cyclomatic Complexity (CC) = 4
- It means the program has 4 linearly independent paths, so at least 4 test cases are required for complete (basis path) testing.
- Basis (independent) paths for the example:

P1 : 1 – 2 – 3 – 4 – 8 (a > b and a > c)
P2 : 1 – 2 – 3 – 7 – 8 (a > b and a ≤ c)
P3 : 1 – 2 – 5 – 6 – 8 (a ≤ b and b > c)
P4 : 1 – 2 – 5 – 7 – 8 (a ≤ b and b ≤ c)

Q6. Software testing terminology: failure, defect, error, testware, test oracle (( ONE TIME ONLY))

ANS

# Software Testing Terminology : Failure, Defect, Error, Testware and Test Oracle (10 Marks)

## 1) Error

- An error is a **human mistake** made by a developer, tester or anyone involved in the software process.
- It can occur during requirements, design, coding or testing.
- It may lead to a defect in the software.

**Example :** Writing wrong condition (e.g., using <= instead of <).

## 2) Defect (Bug)

- A defect is a **fault or flaw** in the software product (e.g., in code, design or document) caused by an error.
- It is a deviation from the specified requirements.
- It exists in the software until it is detected and fixed.

**Example :** The program accepts invalid input due to wrong condition in the code.

## 3) Failure

- A failure is the **incorrect behavior** of the software when it is executed.
- It occurs when a defect in the software is activated during run time.
- It is visible to the user, such as a wrong output, crash or unexpected result.

**Example :** The application crashes or gives a wrong result for a valid input.

## 4) Testware

- Testware refers to all the **artifacts, resources or products** used during testing.
- It helps in planning, designing, executing and evaluating tests.

**Examples of Testware :**

- Test Plan, Test Cases
- Test Data
- Test Scripts (automated)
- Test Environment setup
- Test Results / Reports

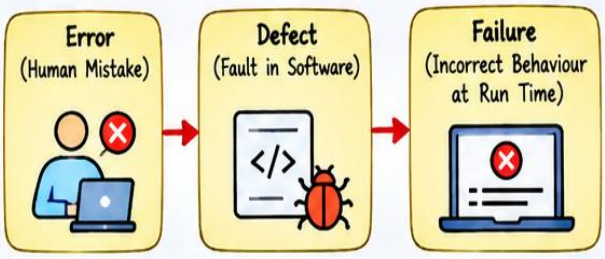
## 5) Test Oracle

- A test oracle is the **source of expected results** used to decide whether the actual output of a test is correct or incorrect.
- It provides the basis for pass or fail of a test case.

**Examples of Test Oracle :**

- Requirement specifications
- Design documents
- Business rules
- Previously verified results
- Manual calculation or domain expert

## Relationship : Error → Defect → Failure




- An Error leads to a Defect, and when the Defect is executed during operation, it may cause a Failure.
- Not every defect results in a failure (it may remain dormant if not executed).

## 6) Summary of Terminology

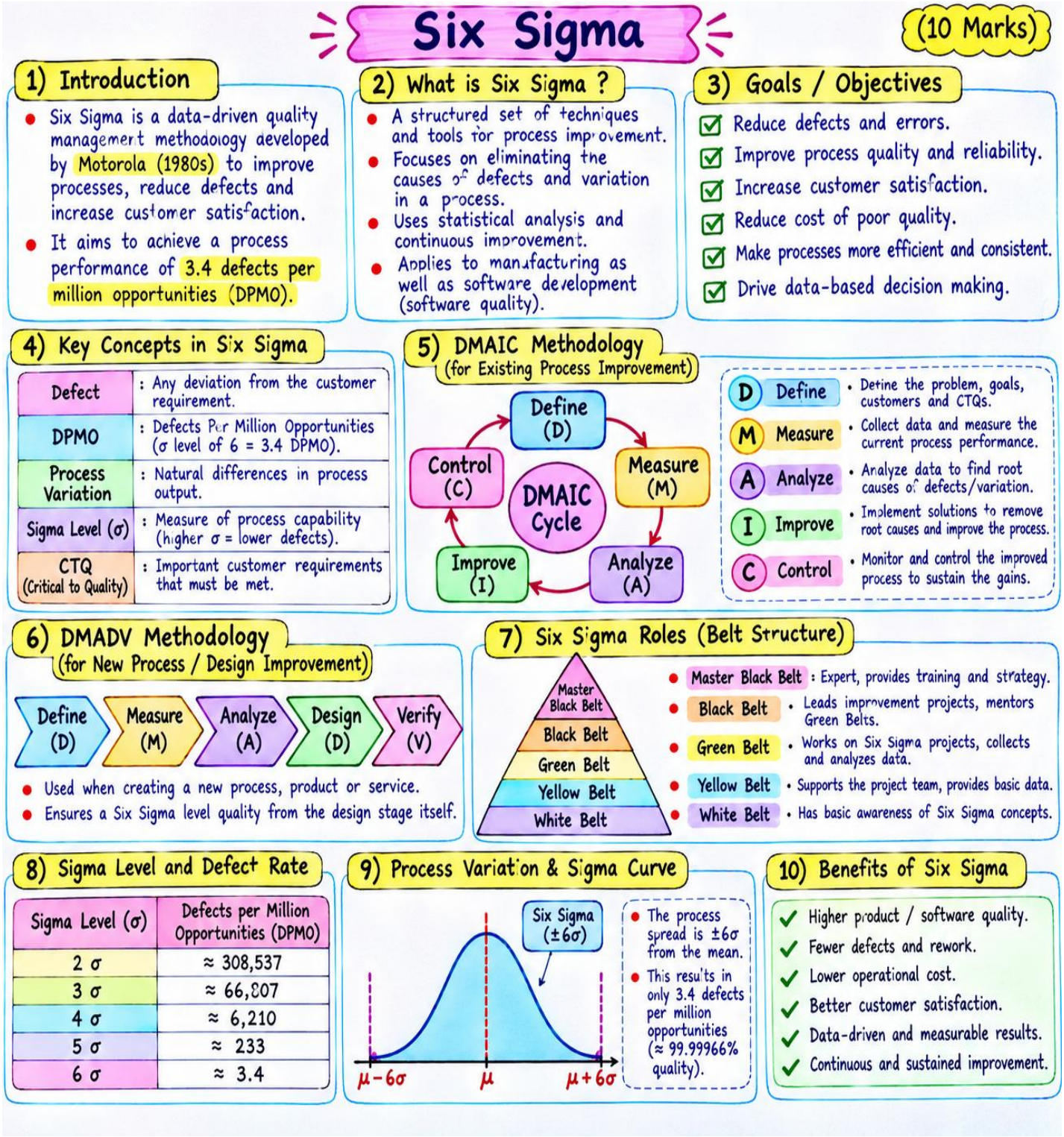
| Term | Meaning (in short) |
|---|---|
| Error | Human mistake in the software process. |
| Defect | Fault / flaw in the software product caused by an error. |
| Failure | Incorrect behavior of the software during execution. |
| Testware | Collection of all testing artifacts and resources (e.g., test plan, test cases, test data, scripts). |
| Test Oracle | Source of expected results to decide pass or fail of a test. |

## 7) Key Differences : Error vs Defect vs Failure

| Aspect | Error | Defect | Failure |
|---|---|---|---|
| Nature | Human mistake | Fault in software | Incorrect behavior at run time |
| Origin | People (developer, tester, analyst) | Introduced by an error (e.g., wrong code) | Occurs when a defect is executed |
| Visibility | Not directly visible in software | Present in the software (code/design/document) | Visible to the user (wrong result, crash) |
| Example | Writing wrong condition | Incorrect condition in code | Program gives wrong output or crashes |

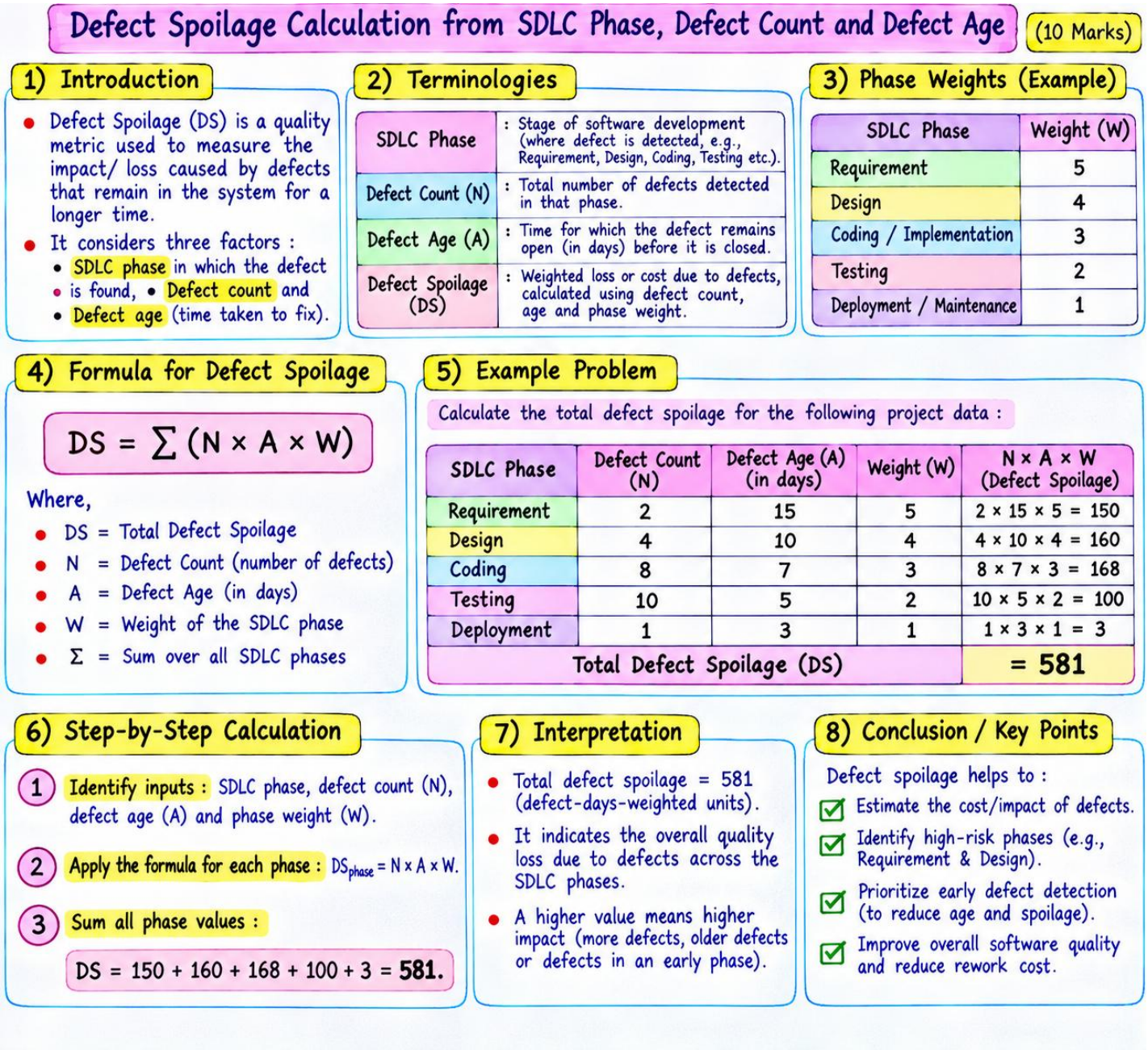
Q7. Six Sigma

ANS



Q8. Defect spoilage calculation from SDLC phase, defect count and defect age.

ANS



Q9. Boundary Value Checking (BVC) and Robust testing test-case design for minimum of three numbers.

ANS

# Boundary Value Checking (BVC) and Robust Testing : Test-Case Design for Minimum of Three Numbers

(10 Marks)

## 1) Introduction

- BVC and Robust Testing are black-box test design techniques used to test boundary values of the input domain.
- Here, we design test cases for the program that finds the minimum of three numbers.

## 2) What is Boundary Value Checking (BVC) ?

- It is a test design technique in which test cases are created using the boundary (edge) values of the input domain.
- It checks the behavior of the program at the minimum, maximum and values just inside the boundaries.

## 3) What is Robust Testing ?

- Robust testing is an extension of BVC.
- Along with boundary values, it also uses values just outside the valid range (invalid values) to check the robustness and error handling of the program.

## 4) Problem Statement & Assumptions

**Program :** Find the minimum of three numbers (A, B, C).

**Assumptions :**

- Each input number is an integer.
- Valid range for each input : 1 to 100. (Min = 1, Max = 100)

## 5) Boundary Values and Test-Value Selection

| Technique | Test Values for Each Input (A, B, C) | Meaning |
|---|---|---|
| BVC (5 values) | { 1, 2, 50, 99, 100 } | Min, Min + 1, Nominal (mid), Max – 1, Max |
| Robust (7 values) | { 0, 1, 2, 50, 99, 100, 101 } | Includes two invalid values (0 and 101) also |

## 6) Test Cases using Boundary Value Checking (BVC)

| TC No. | A | B | C | Expected Result | Purpose / Coverage |
|---|---|---|---|---|---|
| B1 | 1 | 1 | 1 | 1 | All at minimum (lower bound) |
| B2 | 1 | 2 | 100 | 1 | Minimum, next to minimum, maximum |
| B3 | 2 | 50 | 99 | 2 | Just above minimum and intermediate values |
| B4 | 50 | 50 | 50 | 50 | Nominal (middle) values |
| B5 | 99 | 100 | 100 | 99 | Just below and at maximum |
| B6 | 100 | 100 | 100 | 100 | All at maximum (upper bound) |

## 7) Test Cases using Robust Testing

| TC No. | A | B | C | Expected Result | Purpose / Coverage |
|---|---|---|---|---|---|
| R1 | 0 | 50 | 50 | Invalid (Input out of range) | A below minimum (0 < 1) |
| R2 | 101 | 50 | 50 | Invalid (Input out of range) | A above maximum (101 > 100) |
| R3 | 1 | 0 | 50 | Invalid (Input out of range) | B below minimum |
| R4 | 1 | 50 | 101 | Invalid (Input out of range) | C above maximum |
| R5 | 1 | 50 | 99 | 1 | Valid boundary (Min) |
| R6 | 99 | 100 | 1 | 1 | Valid with max and min |
| R7 | 50 | 50 | 50 | 50 | Nominal valid case |

## 8) Conclusion

- BVC verifies the correct behavior at the boundary values of the input domain.
- Robust testing checks both boundary values and invalid (out-of-range) inputs to ensure robustness.
- For the "minimum of three numbers" program, the above test cases provide good coverage of normal, boundary and error conditions.

## Key Takeaways

- ✓ BVC → uses 5 key values (Min, Min+1, Nominal, Max – 1, Max).
- ✓ Robust Testing → uses 7 values (includes Min – 1 and Max+1 also).
- ✓ Both techniques improve the reliability and error handling of the program.

Q10. Need for software measurement and types of software metrics.

ANS

# Need for Software Measurement and Types of Software Metrics (10 Marks)

## 1) Introduction

- Software measurement is the process of quantifying various attributes of a software product, process or project.
- It uses software metrics to collect, analyze and interpret data for better decision-making and quality improvement.

## 2) Need / Importance of Software Measurement

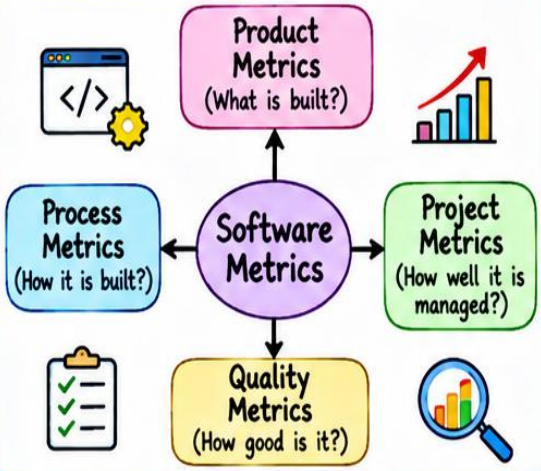
- ☑ To know the size, effort and cost of a software project.
- ☑ To monitor and control project progress.
- ☑ To evaluate quality (reliability, maintainability, etc.).
- ☑ To identify defects early and improve product quality.
- ☑ To make data-driven decisions.
- ☑ To estimate and plan future projects.

## 3) Objectives / Benefits




- Improve product quality and reliability.
- Support estimation (cost, time, resources).
- Track and control the development process.
- Facilitate comparison between projects or releases.
- Provide a basis for continuous process improvement.
- Ensure better management of risks and resources.

## 4) Types of Software Metrics




## 5) Description and Examples of Each Type

| Type of Metric | Meaning | Examples |
|---|---|---|
| Product Metrics | Measure the characteristics of the software product (size, complexity, etc.). | • Size : Lines of Code (LOC)<br>• Complexity : Cyclomatic Complexity |
| Process Metrics | Measure the performance and effectiveness of the software development process. | • Defect Removal Efficiency (DRE)<br>• Code Review Effectiveness |
| Project Metrics | Measure the progress, effort, resources and performance of a software project. | • Effort : Person-Hours<br>• Schedule Variance, Cost Variance |
| Quality Metrics | Measure the quality attributes of the software (reliability, maintainability, usability, etc.). | • Defect Density (defects/KLOC)<br>• Mean Time Between Failures (MTBF) |

## 6) Common Software Metrics (Examples)

- ☑ Size Metric – LOC, Function Points.
- ☑ Complexity Metric – Cyclomatic Complexity.
- ☑ Defect Metric – Defect Density, DRE.
- ☑ Reliability Metric – MTBF, MTTR.
- ☑ Maintainability Metric – Maintainability Index.
- ☑ Productivity Metric – LOC per person per month.

## 7) Example : Defect Density

- It measures the number of defects per thousand lines of code (KLOC).

$$\text{Defect Density} = \frac{\text{Total Defects}}{\text{KLOC}}$$

**Example :**

If a software has 50 defects and 10,000 LOC (i.e. 10 KLOC), then

$$\text{Defect Density} = \frac{50}{10} = 5 \text{ defects/KLOC}$$

## 8) Conclusion

- Software measurement is essential for quantitative and objective evaluation of software products, processes and projects.
- It helps in improving quality, controlling cost and time, and ensuring successful delivery.
- By using different types of software metrics (product, process, project and quality), organizations can achieve continuous improvement and higher customer satisfaction.

Q11. Static vs dynamic testing tools

ANS



Q12. Prime-number program: DD graph, cyclomatic complexity, independent paths and test case

ANS

# Prime-number Program : DD Graph, Cyclomatic Complexity, Independent Paths and Test Case (10 Marks)

## 1) Introduction

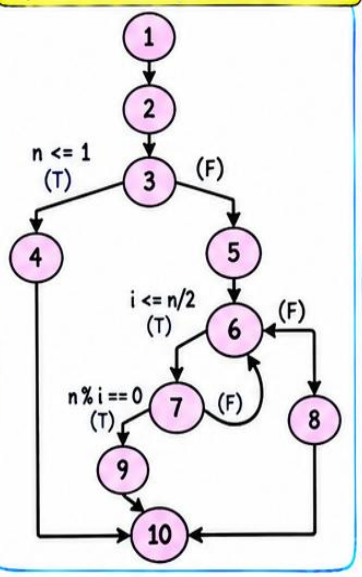
- A prime number is a natural number greater than 1 that has only two factors : 1 and itself.
- In software testing, we can analyze the prime-number program using a Decision Diagram (DD) graph, calculate Cyclomatic Complexity, find Independent Paths and design Test Cases.

## 2) Program : Prime Number

```
int n, i = 2;
boolean flag = true;

if (n <= 1)
    flag = false;
else {
   while (i <= n/2) {
        if (n % i == 0) {
            flag = false;
            break;
        }
        i++;
    }
}
if (flag)
    print("Prime Number");
else
  print("Not a Prime Number");
```

## 3) Decision Diagram (DD) Graph




## 4) Node & Edge Count

| | |
|---|---|
| Number of Nodes (N) | 10 |
| Number of Edges (E) | 11 |
| Number of Predicate (Decision) Nodes (P) | 3 |

## 5) Cyclomatic Complexity (V(G))

- Cyclomatic Complexity (V(G)) gives the number of linearly independent paths in the program.
- It can be calculated as :

$V(G) = E - N + 2$

$V(G) = 11 - 10 + 2 = 3$

(OR) $V(G) = P + 1 = 3 + 1 = 3$

## 6) Independent Paths

- Number of independent paths = V(G) = 3.
- The three linearly independent paths are :

| | |
|---|---|
| Path 1 (P1) | 1 → 2 → 3(T) → 4 → 10 |
| Path 2 (P2) | 1 → 2 → 3(F) → 5 → 6(F) → 8 → 10 |
| Path 3 (P3) | 1 → 2 → 3(F) → 5 → 6(T) → 7(T) → 9 → 10 (Loop in 6-7 is executed at least once) |

## 7) Test Cases

| TC No. | Input (n) | Path Covered | Expected Output | Purpose |
|---|---|---|---|---|
| TC1 | 1 | P1 | Not a Prime Number | Checks for n <= 1 (invalid / non-prime case). |
| TC2 | 2 | P2 | Prime Number | Valid prime number (loop not executed, i <= n/2 is false). |
| TC3 | 4 | P3 | Not a Prime Number | Valid non-prime number (loop executed, n % i == 0 true). |

## 8) Expleandion of Paths

- **P1** : n <= 1 is true, flag = false and the program prints "Not a Prime Number".
- **P2** : n > 1, loop condition (i <= n/2) is false (e.g., n = 2), so no iteration is needed and flag remains true, hence "Prime Number".
- **P3** : n > 1, loop executes and n % i == 0 becomes true (e.g., n = 4, i = 2), flag = false and "Not a Prime Number", is printed.

## 9) Conclusion

- ☑ The DD graph shows the control flow of the prime-number program.
- ☑ Cyclomatic complexity V(G) = 3, hence there are 3 independent paths.
- ☑ The independent paths and corresponding test cases provide adequate coverage to test the program.
- ☑ These test cases check boundary condition, valid prime and valid non-prime cases.

Q13. McCall's quality factors and criteria.

ANS

# McCall's Quality Factors and Criteria

(10 Marks)

## 1) Introduction

- McCall's Quality Model is one of the earliest and widely used models for software quality evaluation.
- It was proposed by Jim McCall (1977) to identify, define and measure the important quality factors of software.

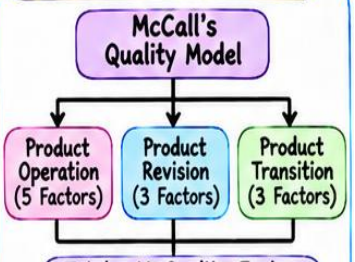
## 2) McCall's Quality Model

- It organizes software quality into 3 perspectives having 11 quality factors and several quality criteria.
- Each factor is measured using one or more criteria that help in assessing the quality of a software product.

## 3) Three Perspectives

- **Product Operation** → How well the software operates in real use.
- **Product Revision** → How easily the software can be changed.
- **Product Transition** → How well the software can adapt to a new environment.

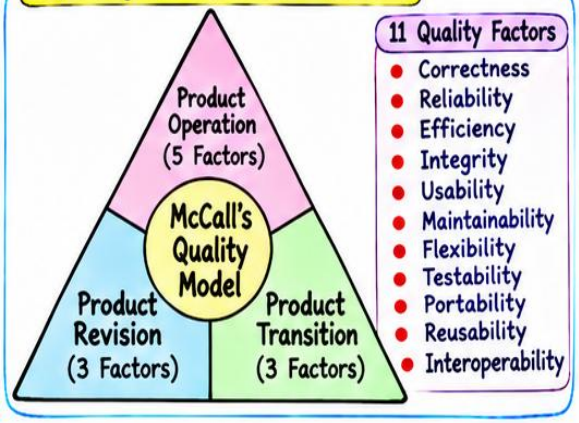
## 4) Model Structure




## 5) McCall's Quality Factors and Their Criteria

| Perspective | Quality Factor (11 in total) | Meaning | Quality Criteria (How it is measured) |
|---|---|---|---|
| Product Operation (5 Factors) | 1. Correctness | Degree to which software meets its requirements. | Completeness, Consistency, Traceability, Accuracy. |
| | 2. Reliability | Ability to perform without failure. | Availability, Fault tolerance, Recoverability, Reliability. |
| | 3. Efficiency | Use of system resources (time, memory, etc.) effectively. | Execution time, Memory usage, Resource utilization. |
| | 4. Integrity | Protection against unauthorized access and data loss. | Access control, Data integrity, Security mechanisms. |
| | 5. Usability | Ease of learning and using the software. | Operational ease, Training effort, User interface quality. |
| Product Revision (3 Factors) | 6. Maintainability | Ease of finding and fixing defects. | Analyzability, Changeability, Stability, Testability. |
| | 7. Flexibility | Ease of modifying the software for new needs. | Adaptability, Extensibility, Configurability. |
| | 8. Testability | Ease of testing the software. | Observability, Controllability, Repeatability. |
| Product Transition (3 Factors) | 9. Portability | Ease of moving the software to another environment. | Adaptability, Installability, Replaceability. |
| | 10. Reusability | Extent to which software components can be reused. | Generality, Modularity, Independence. |
| | 11. Interoperability | Ability to work with other software systems. | Communication compatibility, Data compatibility, Interface compatibility. |

## 6) Diagram : McCall's Model




## 7) Key Points

- ✅ Total 3 perspectives.
- ✅ Total 11 quality factors.
- ✅ Each factor has measurable criteria.
- ✅ Helps in quality evaluation, comparison and improvement of software.
- ✅ Foundation for many later quality models (e.g., ISO 9126, SQA models).

## 8) Conclusion

- McCall's Quality Model provides a structured and comprehensive framework for evaluating software quality.
- It identifies 11 important quality factors and related criteria under 3 perspectives (Product Operation, Product Revision and Product Transition).
- The model helps in building high-quality, reliable, maintainable and adaptable software systems.

Q14. Test plan components and test plan hierarchy.

ANS

# Test Plan Components and Test Plan Hierarchy

(10 Marks)

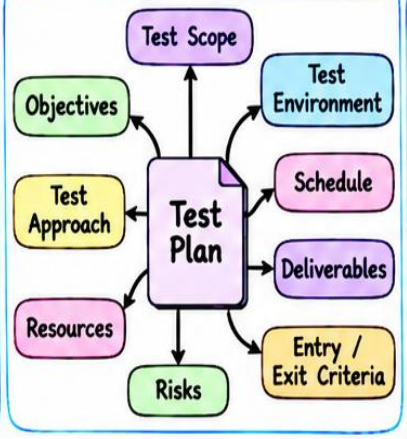
## 1) Introduction

- A **Test Plan** is a formal document that defines the scope, approach, resources, schedule and activities for testing a software product.
- It acts as a **roadmap** for the entire testing process and ensures systematic and effective testing.

## 2) Objectives of Test Plan

- ☑ Define the testing scope and objectives.
- ☑ Plan and organize testing activities.
- ☑ Allocate resources and responsibilities.
- ☑ Estimate effort, time and cost.
- ☑ Specify test deliverables and criteria (entry/exit).
- ☑ Manage and track testing risks.

## 3) Components of Test Plan



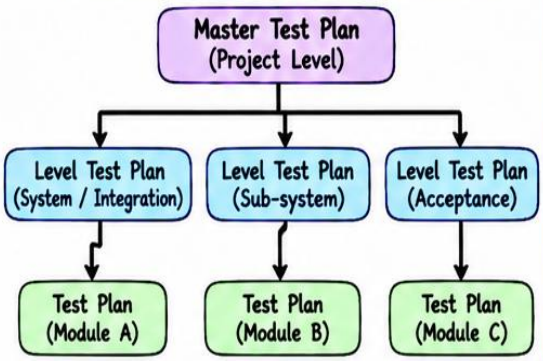

## 4) Components – Details

| Component | Description |
|---|---|
| Objectives | Purpose of testing and goals to be achieved. |
| Test Scope | What will be tested and what is out of scope. |
| Test Approach | Testing strategy, levels, types and techniques. |
| Test Environment | Hardware, software, tools and test data required. |
| Resources | People, roles and responsibilities. |
| Schedule | Timeline and milestones for testing activities. |
| Deliverables | Test cases, test data, test reports, defect report, etc. |
| Entry / Exit Criteria | Conditions to start and complete testing. |
| Risks & Mitigation | Possible testing risks and plans to handle them. |

## 5) Test Plan Hierarchy

- Test plan hierarchy represents the organization of test plans at different levels in a project.
- It shows a **top-down structure** from overall project plan to detailed test plans for each level or module.

**Test Plan Hierarchy (Typical Structure)**




## 6) Levels of Test Plans (Explanation)

| Level | Description |
|---|---|
| Master Test Plan | Overall test plan for the entire project. Defines common strategy, resources and standards. |
| Level Test Plan (System/Integration) | Plan for a specific level of testing (e.g., system testing, integration testing). |
| Sub-system Test Plan | Test plan for a sub-system or major component. |
| Module Test Plan | Detailed test plan for an individual module or unit. |
| Acceptance Test Plan | Plan for user or customer acceptance testing (UAT). |

## 7) Example – Test Plan (Extract)

| | |
|---|---|
| Project | Online Examination System |
| Test Objective | Verify the functionality, performance and security of the system. |
| Test Scope | Login, exam creation, question bank, result generation (out of scope: payment). |
| Test Approach | Use functional, integration and system testing (black-box technique). |
| Environment | Windows 11, Java, MySQL, Selenium. |
| Schedule | 2 weeks (Test design – 3 days, Execution – 6 days, Reporting – 2 days). |
| Exit Criteria | ≥ 95% test cases passed and no critical defects. |

## 8) Conclusion

- ☑ A **Test Plan** defines what, how, when and by whom testing will be done.
- ☑ Its key components include objectives, scope, approach, resources, environment, schedule, deliverables and entry/exit criteria.
- ☑ The test plan hierarchy organizes testing from an overall project plan (master) to detailed plans (system, sub-system, module and acceptance), ensuring better control, coordination and test coverage.



Q15. Bug life cycle and bug states.

ANS

# Bug Life Cycle and Bug States

(10 Marks)

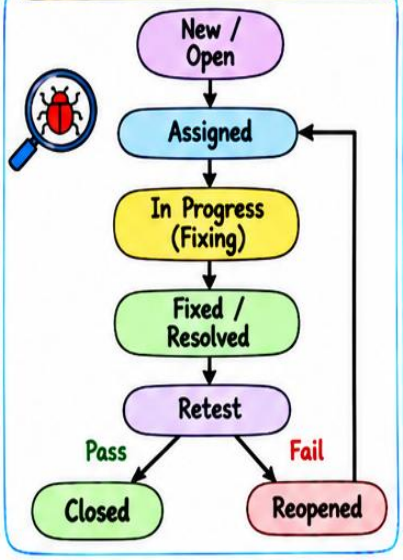
## 1) Introduction

- A **Bug (Defect)** is an error, fault or flaw in the software that causes incorrect or unexpected behavior.
- The **Bug Life Cycle (BLC)** is the sequence of states through which a defect passes from its discovery to final closure.

## 2) Objectives / Need

- ☑ To track and manage defects systematically.
- ☑ To ensure every defect is analyzed, fixed and verified.
- ☑ To maintain quality and reliability of the software.
- ☑ To give clear status of each defect to the team and stakeholders.

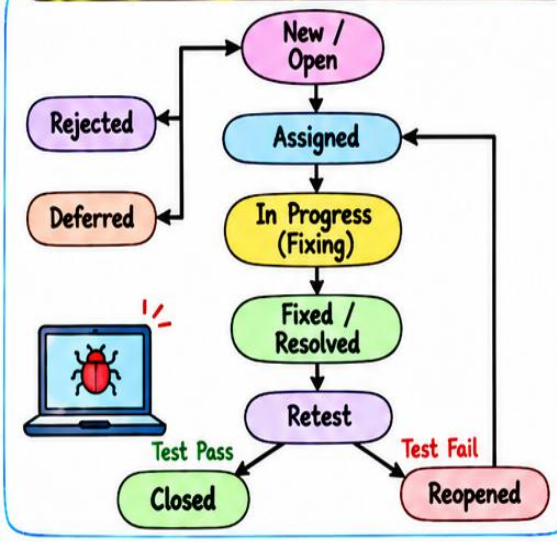
## 3) Bug Life Cycle - Flow Diagram




## 4) Bug States (with Meaning)

| Bug State | Meaning |
|---|---|
| New / Open | Defect is reported and logged in the system. |
| Assigned | Given to a developer for analysis and fixing. |
| In Progress | Developer is working on the defect (fix is under development). |
| Fixed / Resolved | Developer has fixed the defect and marked it as resolved. |
| Retest | Fix is deployed and tested by the tester. |
| Reopened | Defect still exists, so it is reopened and sent back for fixing. |
| Closed | Defect is verified and confirmed as fixed. No further action required. |
| Rejected | Reported issue is invalid, duplicate or not a defect (e.g., expected behavior). |
| Deferred | Fix is postponed to a future release. |

## 5) Complete Bug Life Cycle (with Transitions)




## 6) Bug Life Cycle - Stages (Description)

| Stage | Description |
|---|---|
| 1. Logging | Tester finds a defect and logs it with details (steps, screenshot, severity, etc.). |
| 2. Assignment | Defect is assigned to a developer. |
| 3. Analysis & Fixing | Developer analyzes the defect, identifies root cause and implements the fix. |
| 4. Resolution | Developer marks it as Fixed/Resolved and provides the build for testing. |
| 5. Retesting | Tester verifies the fix in the new build. |
| 6. Closure | If the defect is not reproducible, it is Closed; otherwise it is Reopened. |

## 7) Important Terms

- **Defect / Bug** : Error in the software that needs to be fixed.
- **Severity** : Impact of the defect (e.g., Critical, Major, Minor).
- **Priority** : Order in which the defect should be fixed (e.g., High, Medium, Low).
- **Bug Tracking Tool** : Tool to log and track defects (e.g., JIRA, Bugzilla, Mantis).

## 8) Example - Bug Life Cycle in Practice

| | | |
|---|---|---|
| Scenario | : | Login button is not working. |
| Flow | : | New → Assigned → In Progress → Fixed → Retest → Closed |
| If issue still exist | : | Retest → Reopened → Assigned → ... → Closed |
| If not a defect | : | New → Rejected |
| If fix is later | : | New → Deferred |

## 9) Conclusion



- ☑ The Bug Life Cycle defines the complete flow of a defect from discovery to closure.
- ☑ It includes well-defined bug states such as Open, Assigned, In Progress, Resolved, Retest, Reopened, Closed, etc.
- ☑ It helps in effective defect tracking, timely fixing, better communication and improved software quality.

Q16. Effective vs exhaustive software testing.

ANS

# Effective vs Exhaustive Software Testing (10 Marks)

| Sr. No. | Comparison Point | Effective Software Testing | Exhaustive Software Testing |
|---|---|---|---|
| 1. | Meaning | A practical and smart testing approach that finds maximum defects with optimum effort. | A complete testing approach that tests all possible inputs, combinations and paths in the system. |
| 2. | Objective | To detect critical and high-risk defects and ensure required quality. | To find every possible defect by testing all scenarios without any omission. |
| 3. | Test Coverage | Provides sufficient and risk-based coverage (e.g., high-risk areas, boundary cases). | Provides 100% coverage of all inputs, conditions, combinations and paths. |
| 4. | Test Cases | Uses a limited and selected set of test cases. | Uses all possible test cases (very large in number). |
| 5. | Effort & Time | Less effort and time; feasible in real projects. | Requires very high effort and time; usually impractical. |
| 6. | Cost | Cost-effective and resource-efficient. | Very costly due to huge number of test cases and long execution time. |
| 7. | Feasibility | Practically feasible and widely used in industry. | Generally not feasible for large or complex software. |
| 8. | Applicable Scenarios | Used in real-world projects where time, cost and resources are limited. | Used for small systems, critical modules, or academic / theoretical analysis. |
| 9. | Advantage | Gives high defect detection with optimal use of effort, time and cost. | Ensures maximum confidence and very high reliability in software. |
| 10. | Limitation | May miss some low-risk or rare defects. | Impractical for large systems, leads to huge test suite and resource consumption. |
| 11. | Example | Using equivalence class testing, boundary value analysis, and risk-based testing to select test cases. | Testing all possible input values for a function, e.g., for input (0-9), testing 0,1,2,3,4,5,6,7,8,9. |
| 12. | Suitability | Suitable for most software projects as it balances quality, time and cost. | Suitable only for small, simple and safety-critical software where complete testing is essential. |

Q17. Bottom-up integration testing.

ANS

# Bottom-Up Integration Testing (10 Marks)

## 1) Introduction

- Integration Testing is a level of testing in which individually tested modules are combined and tested as a group.
- It checks the interface and interaction between integrated modules.

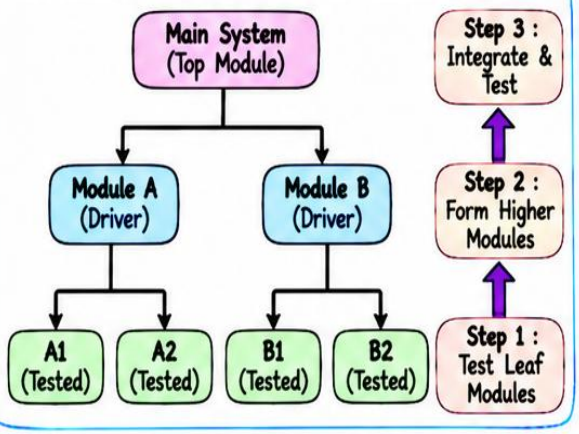
## 2) Definition

→ **Bottom-Up Integration Testing** is an integration testing approach in which the lower-level (leaf) modules are tested first and then progressively integrated upwards to form higher-level modules and finally the complete system.

## 3) Working / Procedure

| | |
|---|---|
| Step 1 → | Test all low-level (leaf) modules individually. |
| Step 2 → | Integrate these modules into higher-level modules. |
| Step 3 → | Use **drivers** to simulate the higher-level modules (if they are not yet available). |
| Step 4 → | Continue integration and testing in an upward direction. |
| Step 5 → | Finally, test the complete integrated system. |

## 4) Diagram : Bottom-Up Integration




## 5) Example

- Consider a system with modules : A1, A2, B1, B2 (lower level) and A, B (higher level) and a Main Module.

**Testing Order (Bottom-Up):**

1. Test A1, A2, B1, B2.
2. Integrate A1 + A2 → Test A.
3. Integrate B1 + B2 → Test B.
4. Integrate A + B → Test Main System.

## 6) Advantages

- ☑ Finds and fixes defects early in low-level modules.
- ☑ No need to wait for the top module to be ready.
- ☑ Easy fault localization (defects are found at lower level).
- ☑ Allows parallel testing of different modules.
- ☑ Improves the reliability of the system step by step.

## 7) Disadvantages

- ☒ Requires the use of drivers, which need extra effort to develop.
- ☒ Does not test the top-level logic in the early stages.
- ☒ Integration is done in many steps, which can be time-consuming.
- ☒ Overall system behaviour is verified only at the end.

## 8) Bottom-Up vs Top-Down Integration

| Aspect | Bottom-Up Integration | Top-Down Integration |
|---|---|---|
| Starting Point | Begins from lower-level modules. | Begins from the top-level module. |
| Test Aids | Uses Drivers. | Uses Stubs. |
| Defect Detection | Early detection in low-level modules. | Early detection in high-level modules. |
| Availability | Top module not required initially. | Top module required at the start. |
| Use Case | When lower modules are stable and available. | When system architecture is well defined. |

## 9) Conclusion

- ☑ Bottom-Up Integration Testing is a systematic and effective approach to integrate and test software modules.
- ☑ It is suitable when lower-level modules are ready and stable, and it helps in finding integration defects early.

Q18. Testing a simple loop

ANS

# Testing a Simple Loop

(10 Marks)

## 1) Introduction

- **Loop Testing** is a type of white-box testing used to check the correctness, boundaries and behaviour of loops in a program.
- A **simple loop** executes a set of statements repeatedly based on a condition.

## 2) Objectives

- ☑ To verify correct execution of the loop.
- ☑ To check loop boundary conditions (0, 1, n, n-1).
- ☑ To ensure proper termination of the loop.
- ☑ To detect errors like infinite loop, off-by-one errors, etc.

## 3) Loop under Test (Example)

**Program : Sum of first n numbers**

```
int sum = 0;
for (int i = 1; i ≤ n; i++) {
    sum = sum + i;
}
return sum;
```

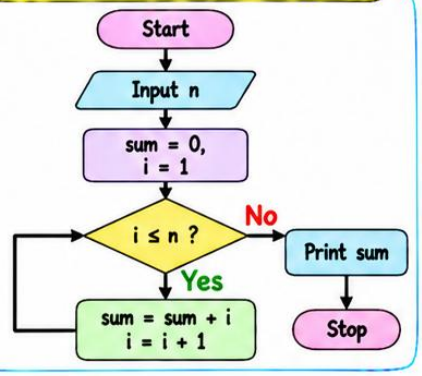
Loop: for (i = 1 to n)
It executes n times.

## 4) Loop Testing Strategy

For a simple loop, the standard test cases are:

1. Zero iteration : n = 0 (loop body not executed)
2. Single iteration : n = 1 (loop executes once)
3. Typical iteration : n = a value between 1 and max (e.g., 5)
4. Boundary iteration : n = n-1 (just below upper limit)
5. Extreme iteration : n = n (maximum allowed value, e.g., 10)

## 5) Flowchart of the Program




## 6) Test Cases for the Loop

| TC No. | Input (n) | Expected Output (sum) | Purpose / Description |
|---|---|---|---|
| TC1 | 0 | 0 | Zero iteration (loop not executed) |
| TC2 | 1 | 1 | Single iteration |
| TC3 | 5 | 15 (1+2+3+4+5) | Typical iteration (normal case) |
| TC4 | 9 | 45 (1+...+9) | Boundary iteration (n = n - 1, if max = 10) |
| TC5 | 10 | 55 (1+...+10) | Extreme iteration (maximum allowed value) |

## 7) Loop Testing - Coverage

| Coverage Aspect | Achieved ? |
|---|---|
| 0 iteration (n = 0) | ✓ |
| 1 iteration (n = 1) | ✓ |
| Typical case (1 < n < max) | ✓ |
| Boundary case (n = n - 1) | ✓ |
| Extreme case (n = max) | ✓ |

## 8) Explanation of Test Cases

| Test Case | Explanation |
|---|---|
| Zero Iteration | Checks the case when the loop body is not executed (n = 0). |
| Single Iteration | Verifies correct execution for one iteration (n = 1). |
| Typical Iteration | Tests the loop for a normal value (e.g., n = 5) to check general functionality. |
| Boundary Iteration | Tests the upper edge - 1 (n = 9) to detect off-by-one errors. |
| Extreme Iteration | Tests the maximum allowed value (n = 10) to ensure correct termination and no overflow. |

## 9) Key Points

- ☑ Loop testing focuses on correctness, boundary and termination of loops.
- ☑ For a simple loop, minimum 5 test cases are used: 0, 1, typical, n - 1 and n.
- ☑ It helps to detect errors like off-by-one error, infinite loop and wrong condition.
- ☑ This technique can be applied to for, while and do-while loops.

## 10) Conclusion

- Testing a simple loop ensures that the loop executes the correct number of times for all boundary and typical cases.
- Using a systematic set of test cases (0, 1, typical, n - 1 and n) provides good coverage and improves the reliability of the program.

Q19. ISO 9000:2000 standard.

ANS

# ISO 9000 : 2000 Standard

(10 Marks)

## 1) Introduction

- ISO 9000:2000 is a set of international standards for Quality Management System (QMS).
- It was released by ISO (International Organization for Standardization) in the year 2000.

## 2) Objectives

- ☑ To provide a common framework for QMS.
- ☑ To improve product and service quality.
- ☑ To enhance customer satisfaction.
- ☑ To ensure continuous improvement.
- ☑ To make quality standards applicable to all types of organizations.

## 3) ISO 9000 Family (2000 Series)

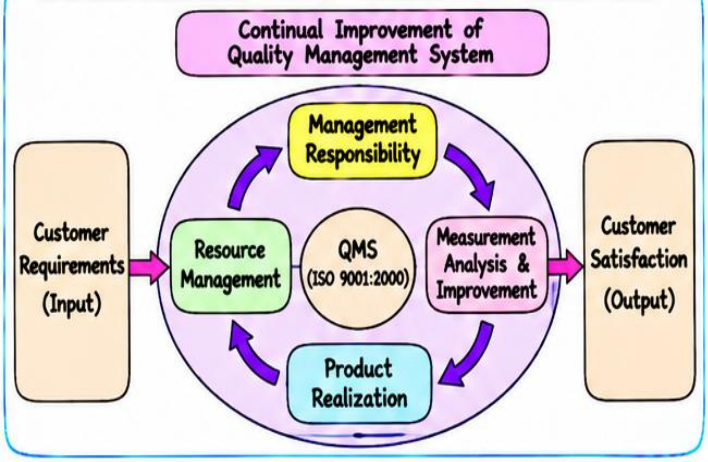
ISO 9000 : 2000 (Family of Standards)

- ISO 9000:2000 (Fundamentals and Vocabulary)
- ISO 9001:2000 (Requirements for QMS) (Certifiable)
- ISO 9004:2000 (Guidelines for Performance Improvement)

## 4) Features of ISO 9000:2000

- Process-based Quality Management System.
- Applies to all types of organizations (software, manufacturing, services etc.).
- Focuses on customer satisfaction.
- Emphasizes continuous improvement.
- More flexible and simpler than earlier versions (1994).
- Promotes prevention of defects (rather than only detection).
- Uses the PDCA (Plan-Do-Check-Act) cycle.

## 5) ISO 9001:2000 – QMS Model (Process Approach)




## 6) Clauses / Requirements of ISO 9001:2000

| Clause | Requirement (Brief Description) |
|---|---|
| 4. Quality Management System | General requirements & documentation (Quality Manual, procedures, records). |
| 5. Management Responsibility | Top management commitment, quality policy, objectives and reviews. |
| 6. Resource Management | Provision and management of resources (people, infrastructure, work environment). |
| 7. Product Realization | Processes for developing and delivering the product / software (e.g., design, development, verification, validation, delivery). |
| 8. Measurement, Analysis & Improvement | Monitoring, measurement, internal audit, control of non-conformities and continual improvement. |

## 7) Benefits of ISO 9000:2000

- ☑ Improves product and service quality.
- ☑ Increases customer confidence and satisfaction.
- ☑ Provides a systematic and process-oriented approach.
- ☑ Reduces errors, rework and cost.
- ☑ Enhances organizational efficiency.
- ☑ Facilitates international recognition and business opportunities.

## 8) ISO 9000:2000 vs ISO 9000:1994

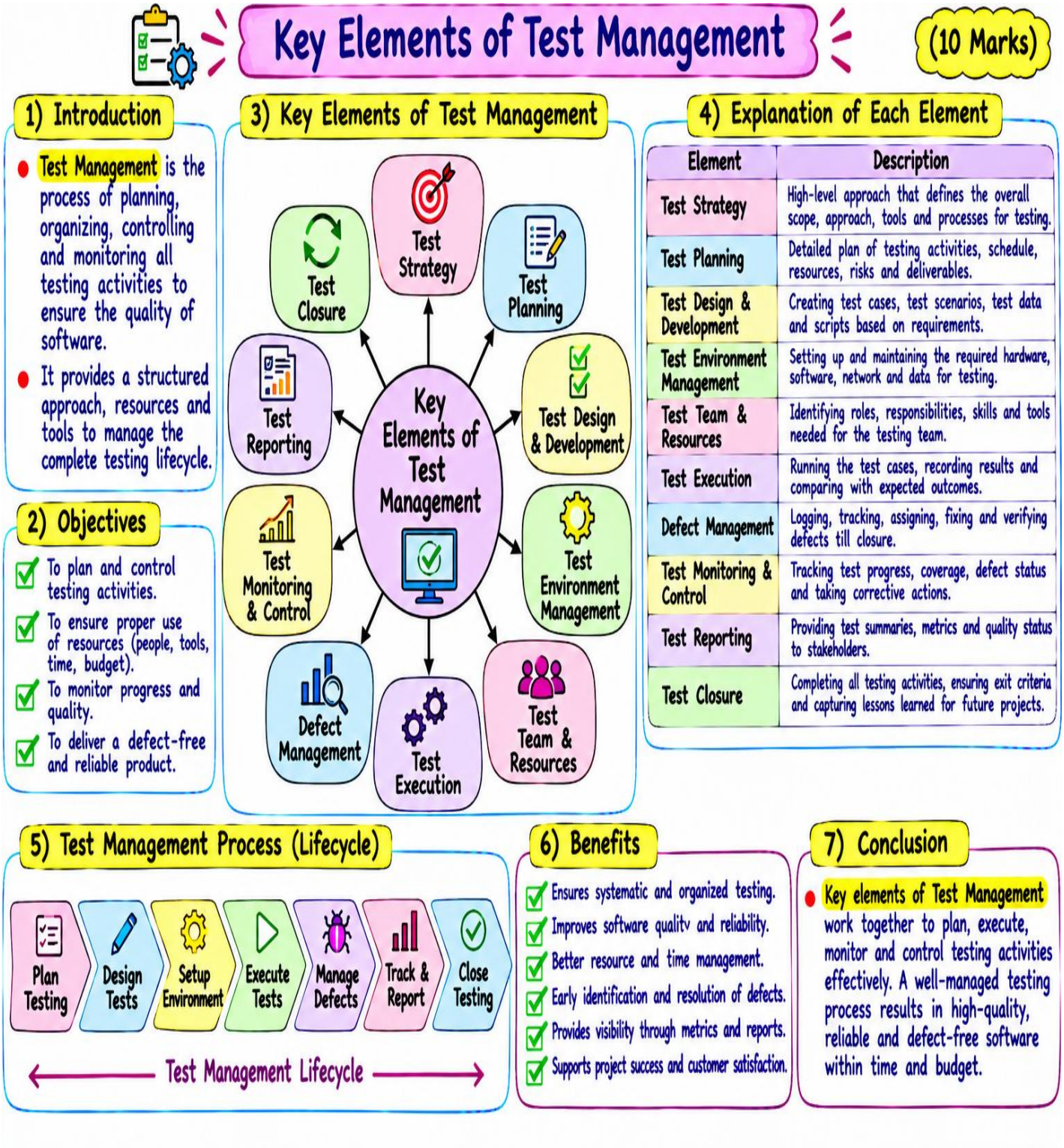
| Aspect | ISO 9000:1994 | ISO 9000:2000 |
|---|---|---|
| Approach | Procedure based, more prescriptive. | Process based, more flexible. |
| Focus | Quality assurance during production. | Overall quality management and customer satisfaction. |
| Applicability | Less generic. | Generic, applies to all types of organizations. |
| Structure | Separate standards (9001, 9002, 9003). | Unified standard (ISO 9001:2000). |
| Improvements | Less emphasis on continual improvement. | Strong focus on continual improvement (PDCA). |

## 9) Conclusion

- ☑ ISO 9000:2000 provides a universal and effective framework for establishing, implementing and continually improving a Quality Management System (QMS).
- ☑ It helps organizations to deliver quality products/services, achieve customer satisfaction and maintain global competitiveness.

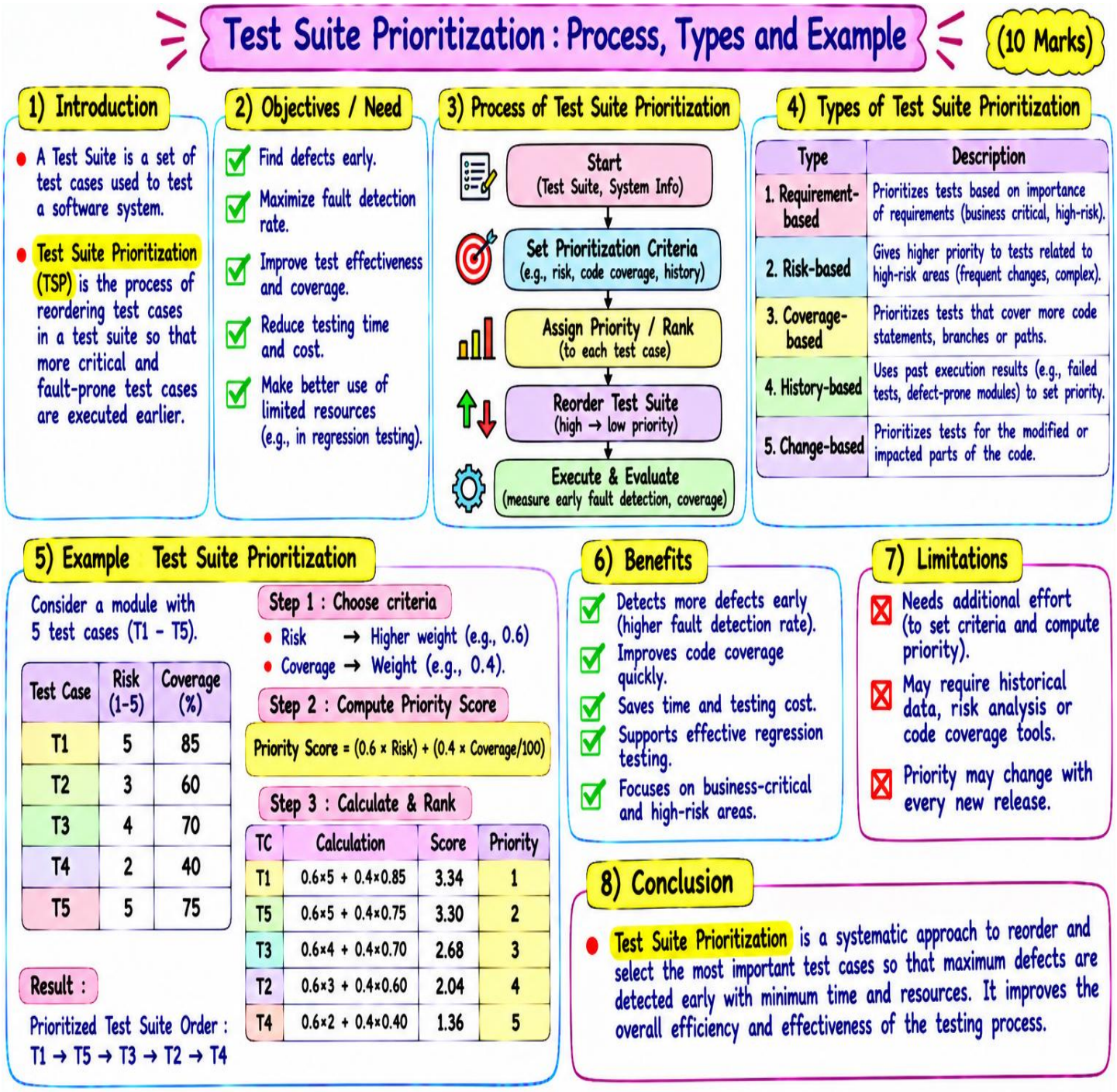
Q20. Key elements of test managemen

ANS



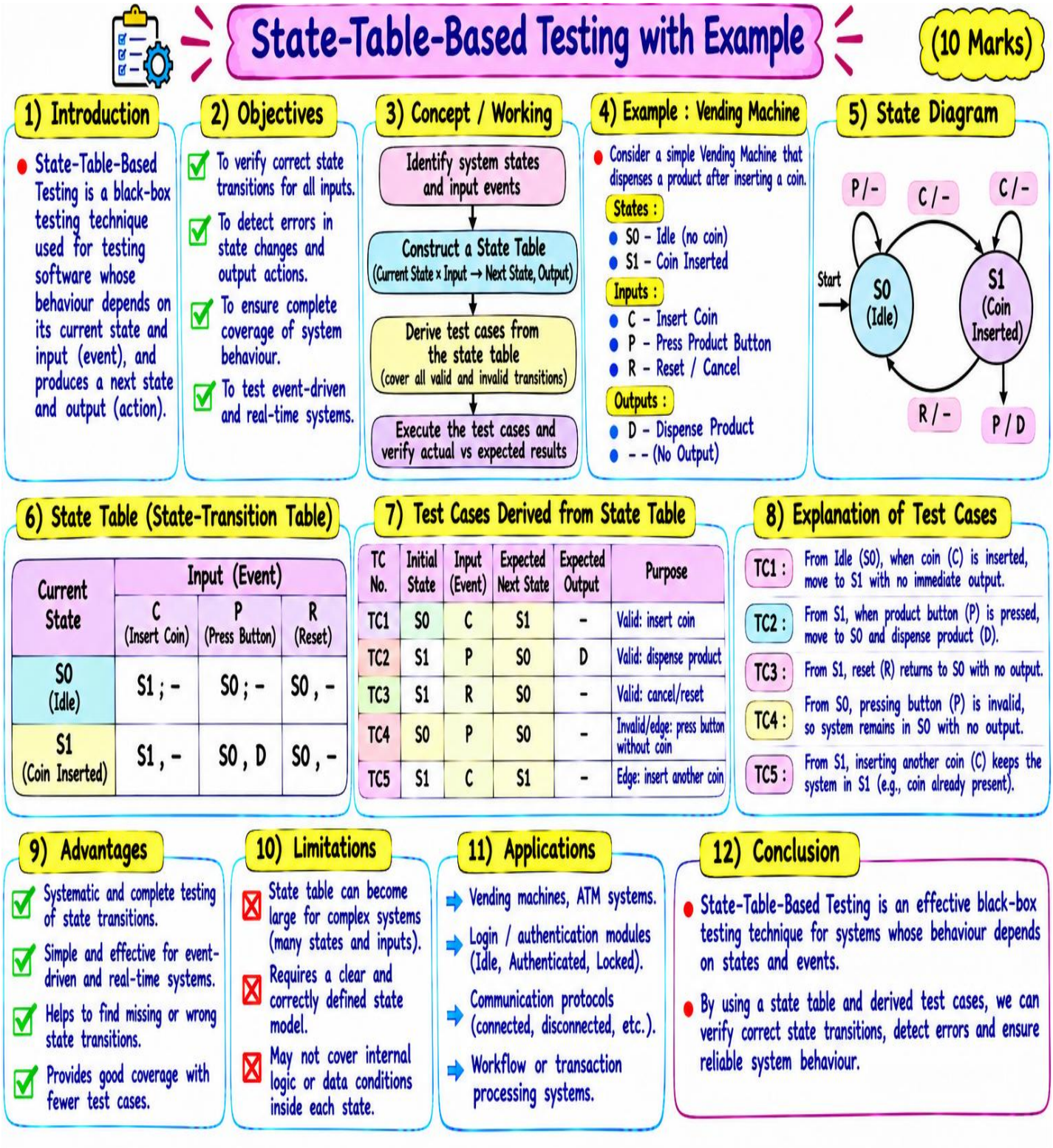
Q21. Test suite prioritization: process, types and example.

ANS



Q22. State-table-based testing with example.

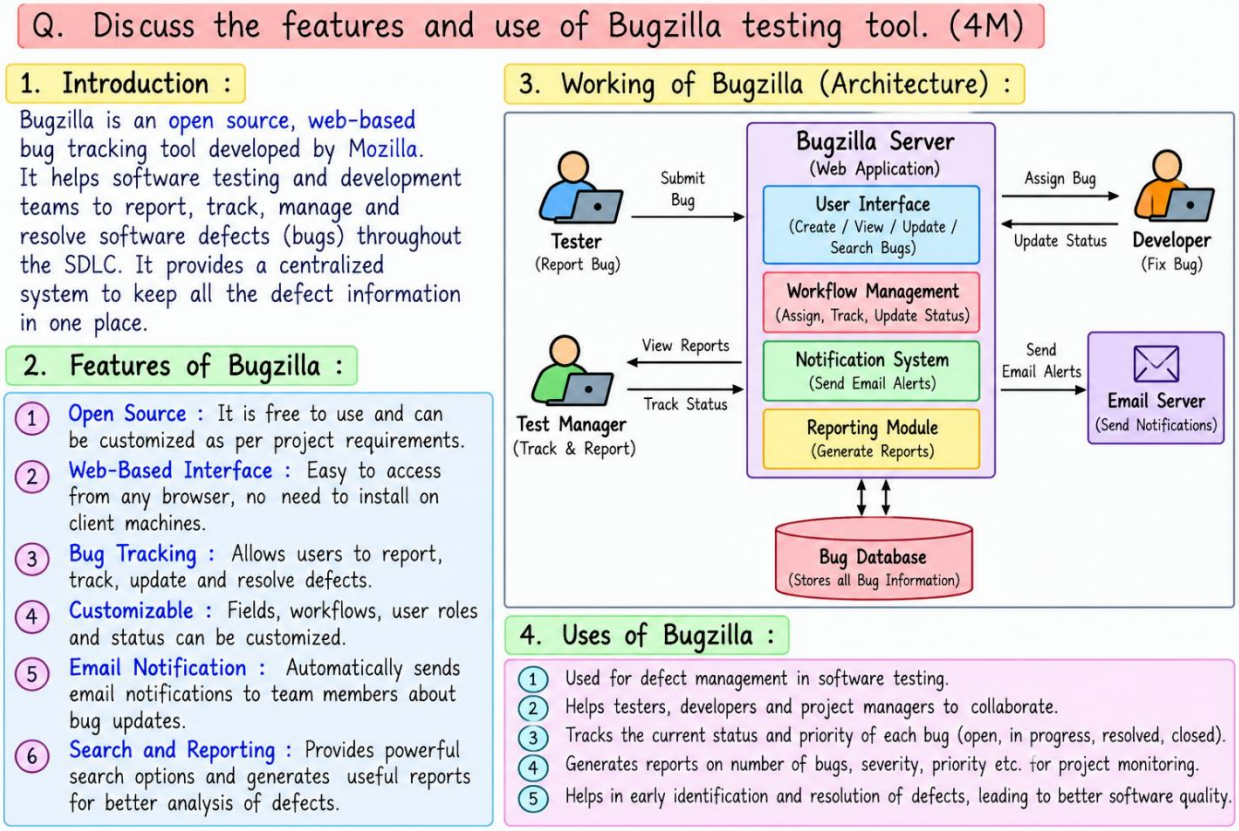
ANS



UT-2

Q1. Discuss the features and use of Bugzilla testing tool. (4M)

ANS




**Theory Answer**

**Bugzilla** is an **open-source, web-based bug/defect tracking tool** developed by Mozilla. It is used by software testing and development teams to **report, track, assign, update and resolve defects** during the Software Development Life Cycle (SDLC).

**Features of Bugzilla**

1. **Open Source:** Free to use and can be customized.
2. **Web-Based:** Can be accessed through a web browser.
3. **Bug Tracking:** Allows testers to report, assign, update and close bugs.
4. **Customizable:** Bug fields, workflows, user roles and statuses can be customized.

5. **Email Notifications:** Sends notifications when a bug is created or updated.
6. **Search & Reporting:** Provides searching and generates useful bug reports.

**Uses of Bugzilla**

- Used for **defect management**.
- Helps **testers and developers collaborate**.
- Tracks the **current status and priority** of bugs.
- Generates reports for project monitoring.
- Helps in **early identification and resolution of defects**, improving software quality.

Q2. Explain Bottom up Integration Testing with one example. (4M)

ANS

## Q. Explain Bottom-up Integration Testing with one example. (4M)

### 1. What is Bottom-up Integration Testing ?

Bottom-up Integration Testing is an integration testing approach in which testing starts from the lowest level modules (leaf or child modules) and gradually integrates higher level modules until the complete system is tested.
In this approach, lower level modules are tested first, and then they are combined to form higher level modules step by step.

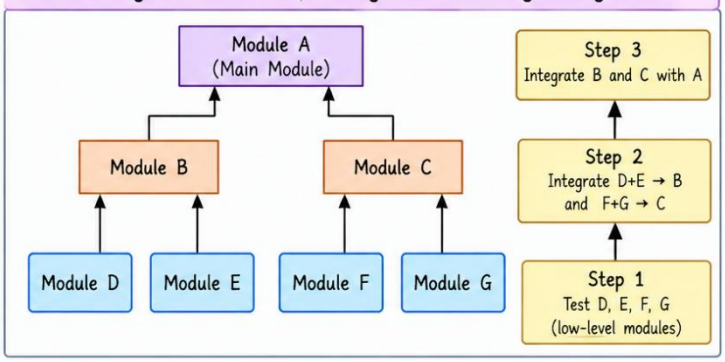
### 2. Steps in Bottom-up Integration Testing :

1. Test individual low-level modules (leaf modules).
2. Integrate related low-level modules and test.
3. Move upward and integrate higher level modules one by one.
4. Repeat the process until the complete system is tested.

### 3. Advantages :

1. No need for stubs.
2. Low-level modules are tested early.
3. Easier to locate and fix defects in lower modules.
4. Helps in building and testing the system step by step.

### 4. Working of Bottom-up Integration Testing (Diagram) :




### 5. Example : Library Management System

Consider a simple Library Management System with the following modules as shown in the diagram above:

- D: Book Search
- E: Member Validation
- F: Issue Book
- G: Return Book
- B: Book Management (uses D and E)
- C: Transaction Management (uses F and G)
- A: Library Management System (main module)

Testing process (Bottom-up):

1. Test modules D, E, F, G individually.
2. Integrate D and E to test module B.
3. Integrate F and G to test module C.
4. Finally, integrate B and C with module A and test the complete system.

**Definition**

**Bottom-Up Integration Testing** is an integration testing technique in which testing starts from the **lowest-level modules** and gradually moves upward by combining them with higher-level modules.

In this approach, the low-level modules are tested first. Then, related modules are integrated and tested step-by-step until the **complete system** is tested.

**Steps**

1. Test the **lowest-level modules** individually.
2. Combine the tested low-level modules and test them together.
3. Move upward and integrate the **higher-level modules**.
4. Continue the process until the **main module and complete system** are tested.

**Example – Library Management System**

Consider the following modules:

- **D:** Book Search
- **E:** Member Validation
- **B:** Book Management
- **F:** Issue Book
- **G:** Return Book
- **C:** Transaction Management
- **A:** Main Library Management System

**Testing sequence:**

**D + E → B**
**F + G → C**
**B + C → A**

Thus, testing starts with **D, E, F and G** and gradually moves upward until the complete **Library Management System (A)** is tested.

**Advantages**

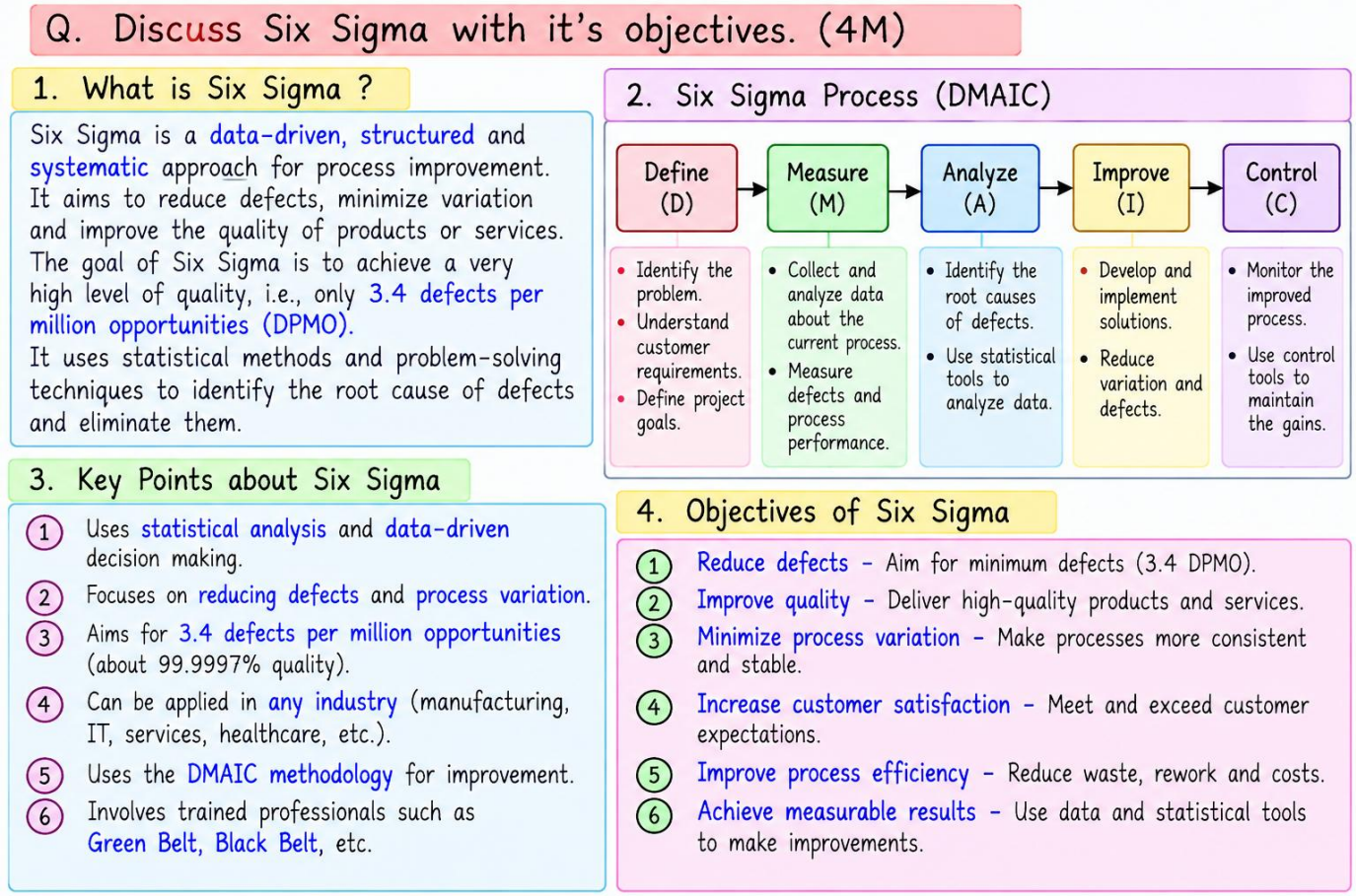
- No need for **stubs**.
- Low-level modules are tested early.
- Defects are easier to locate and fix.
- System is built and tested **step-by-step**.

**Conclusion**

Bottom-Up Integration Testing is useful when the **lower-level modules are stable and important**, as it allows developers and testers to verify the system gradually from the bottom to the top.

Q.3 Discuss Six Sigma with it's objectives. (4M)

ANS




**Definition of Six Sigma**

**Six Sigma** is a **data-driven and systematic approach** used for improving the quality of a process, product or service. It focuses on **reducing defects, errors and process variation**.

The main goal of Six Sigma is to achieve approximately **3.4 defects per million opportunities (DPMO)**.

**2. Six Sigma Process – DMAIC**

Six Sigma commonly uses the **DMAIC** methodology:

**D → Define → M → Measure → A → Analyze → I → Improve → C → Control**

- **Define:** Identify the problem and customer requirements.
- **Measure:** Collect data and measure the current process.
- **Analyze:** Find the root causes of defects.

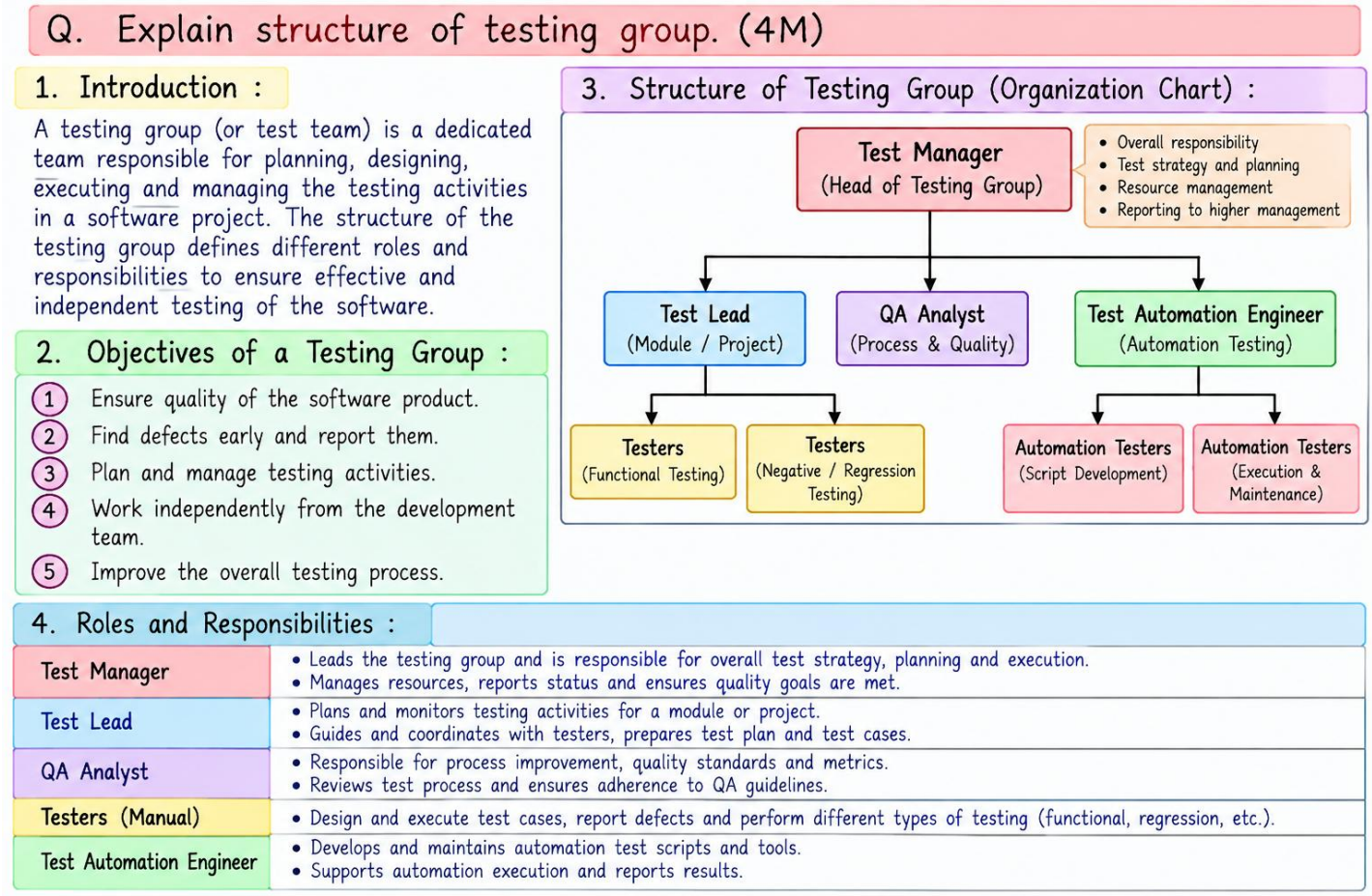
- **Improve:** Implement solutions to reduce defects and variation.
- **Control:** Monitor the improved process and maintain the quality.

**3. Objectives of Six Sigma**

1. **Reduce defects:** Minimize errors and defects in the process.
2. **Improve quality:** Produce reliable and high-quality products/software.
3. **Reduce process variation:** Make the process more consistent and stable.
4. **Increase customer satisfaction:** Meet customer requirements and expectations.
5. **Improve efficiency:** Reduce waste, rework, time and cost.
6. **Achieve measurable improvement:** Use data and statistical techniques for decision-making.

Q4. Explain structure of testing group. (4M)

ANS




**Introduction**

A **Testing Group** or **Test Team** is a group of people responsible for **planning, designing, executing and managing software testing activities**. The team works to identify defects and ensure that the software meets the required quality standards.

**2. Structure of Testing Group**

A typical testing group consists of the following members:

**Test Manager → Test Lead → Testers / QA Analysts / Automation Testers**

**3. Roles and Responsibilities**

**1. Test Manager**

- Leads the entire testing team.
- Prepares the overall **test strategy and test plan**.

- Manages testing resources and schedules.
- Reports testing progress to management.

**2. Test Lead**

- Manages testing activities for a particular project or module.
- Assigns work to testers.
- Reviews test cases and monitors test execution.

**3. QA Analyst**

- Ensures that testing follows **quality standards and processes**.
- Identifies areas for process improvement.
- Monitors quality metrics and documentation.

**4. Testers**

- Design and execute **test cases**.
- Identify and report software defects.
- Perform functional, regression and other types of testing.

**5. Test Automation Engineer**

- Develops and maintains **automated test scripts**.
- Executes automated tests and analyzes results.
- Helps reduce manual testing effort.

Q4. Compare Traditional software testing and web based software testing. (4M)

ANS

## Q. Compare Traditional software testing and web based software testing. (4M)

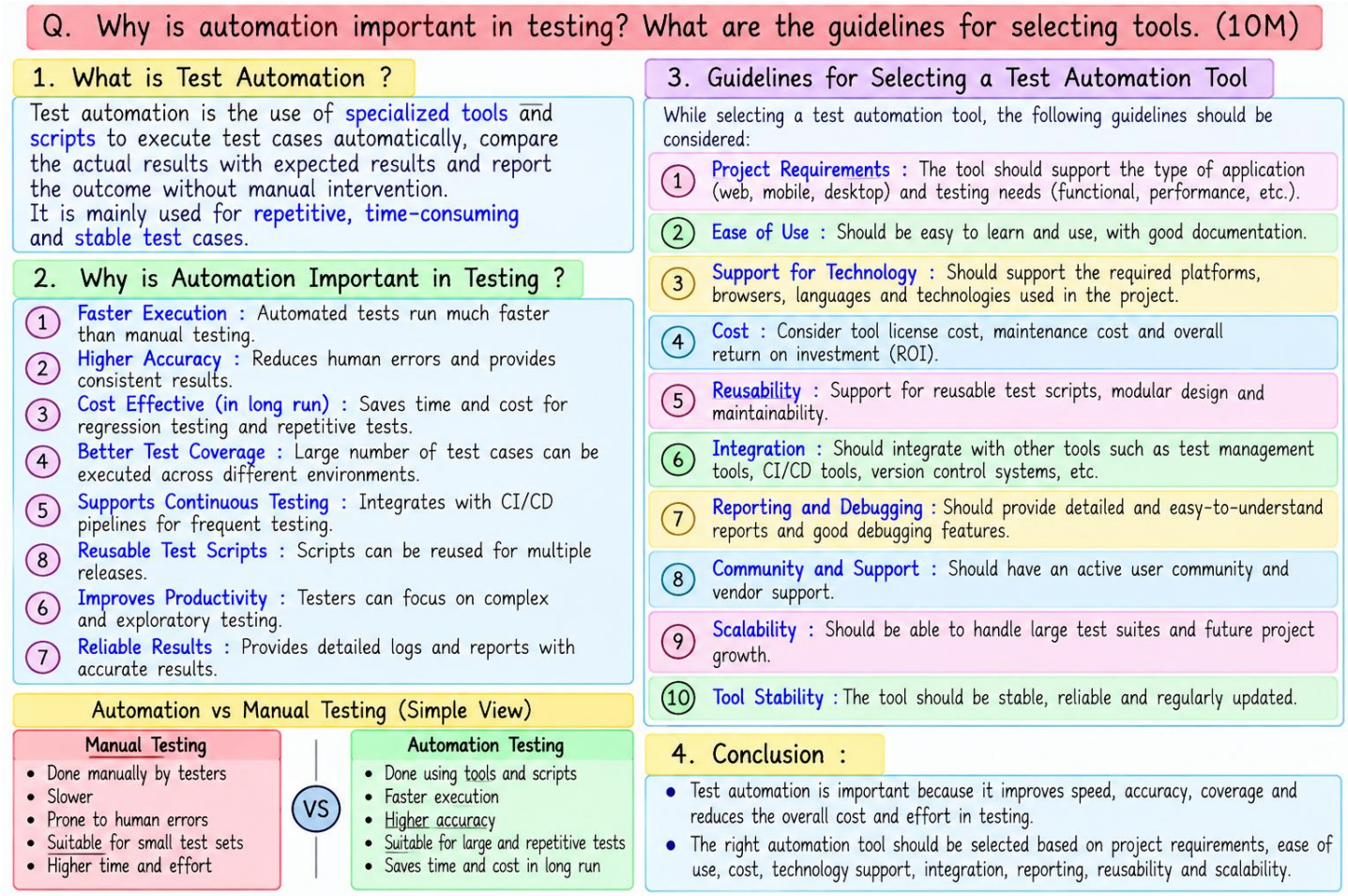
| Aspect | Traditional Software Testing | Web Based Software Testing |
|---|---|---|
| Definition | Testing of stand-alone software applications that are installed on a local system. | Testing of web applications that run on a web server and are accessed through a browser over a network (Internet/Intranet). |
| Platform | Desktop or local environment. | Client-server environment (accessed using browsers like Chrome, Firefox, Edge). |
| Accessibility | Used by a limited number of users on a local machine. | Accessible by multiple users from anywhere using a web browser. |
| Testing Focus | Focuses on functional, performance, and compatibility testing of the installed application. | Includes functional testing as well as additional testing like cross-browser, cross-platform, network, security and usability testing. |
| Test Environment | Controlled and stable environment. | Dynamic and real-time environment (affected by network, server load, etc.). |
| Types of Testing | Functional, performance, compatibility, installation, etc. | Functional, usability, compatibility (cross-browser/ cross-device), security, performance, interface and link testing, etc. |
| Tools | Tools like Selenium (for desktop), JUnit, TestComplete, etc. | Tools like Selenium WebDriver, JMeter, Postman, BrowserStack, etc. |
| Examples | Desktop applications like MS Office, media players, accounting software, etc. | Web applications like Gmail, Amazon, online banking systems, e-commerce websites, etc. |

**Conclusion :**

- Traditional software testing is suitable for stand-alone desktop applications with a controlled environment.
- Web based software testing is more complex as it involves testing over a network, multiple browsers, devices and security aspects.
- Web testing includes all traditional testing activities plus additional testing such as cross-browser, cross-platform, network and security testing.
- Both aim to ensure that the software works correctly and meets user requirements, but web based testing covers a wider range of scenarios.

Q5. Why is automation important in testing? What are the guidelines for selecting tools. (8M)

ANS




### Introduction

**Test Automation** is the use of software tools and scripts to **execute test cases automatically**, compare actual results with expected results, and generate test reports.

It is mainly useful for **repetitive, time-consuming and frequently executed tests**, such as regression testing.

### 2. Why is Automation Important in Testing?

#### 1. Faster Test Execution

Automated test cases can be executed **much faster** than manual testing. A large number of test cases can be run in less time.

#### 2. Reduces Human Errors

Automation reduces mistakes caused by **manual data entry, repetition or lack of concentration**, giving more consistent results.

**3. Better Test Coverage**

Automation allows testers to execute a **large number of test cases** across different browsers, devices and environments.

**4. Useful for Regression Testing**

Whenever new changes are made to software, automated regression tests can be executed repeatedly to check whether existing functionality is still working.

**5. Saves Time and Cost**

Although automation requires an initial investment, it **reduces testing time and effort** in the long run, especially for large projects.

**6. Reusable Test Scripts**

Once created, automated test scripts can be **reused for different releases and test cycles** with minor changes.

**7. Supports Continuous Testing**

Automation can be integrated with **CI/CD pipelines**, allowing tests to run automatically whenever new code is added or deployed.

**8. Improves Productivity**

Testers spend less time performing repetitive tasks and can focus on **complex, exploratory and usability testing**.

**9. Accurate Reporting**

Automation tools can generate **test logs, screenshots, execution results and detailed reports**, making defect analysis easier.

**3. Guidelines for Selecting Test Automation Tools**

The following points should be considered while selecting an automation tool:

**1. Project Requirements**

The tool should match the **type of application and testing requirements**, such as web, mobile or desktop testing.

**2. Ease of Use**

The tool should be **easy to learn, use and maintain**. Good documentation and tutorials are also important.

**3. Technology Support**

The tool should support the required **programming languages, browsers, operating systems and technologies** used in the project.

**4. Cost**

Consider the **license cost, maintenance cost, training cost** and overall return on investment (ROI).

**5. Reusability**

The tool should support **reusable and modular test scripts** so that scripts can be used in future testing cycles.

**6. Integration**

It should integrate easily with other tools such as **test management tools, CI/CD tools, defect tracking systems and version-control systems**.

**7. Reporting and Debugging**

The tool should provide **clear test reports, logs, screenshots and debugging facilities** to easily identify failures.

**8. Scalability**

The tool should be capable of handling **large test suites** and should support future growth of the project.

**9. Support and Community**

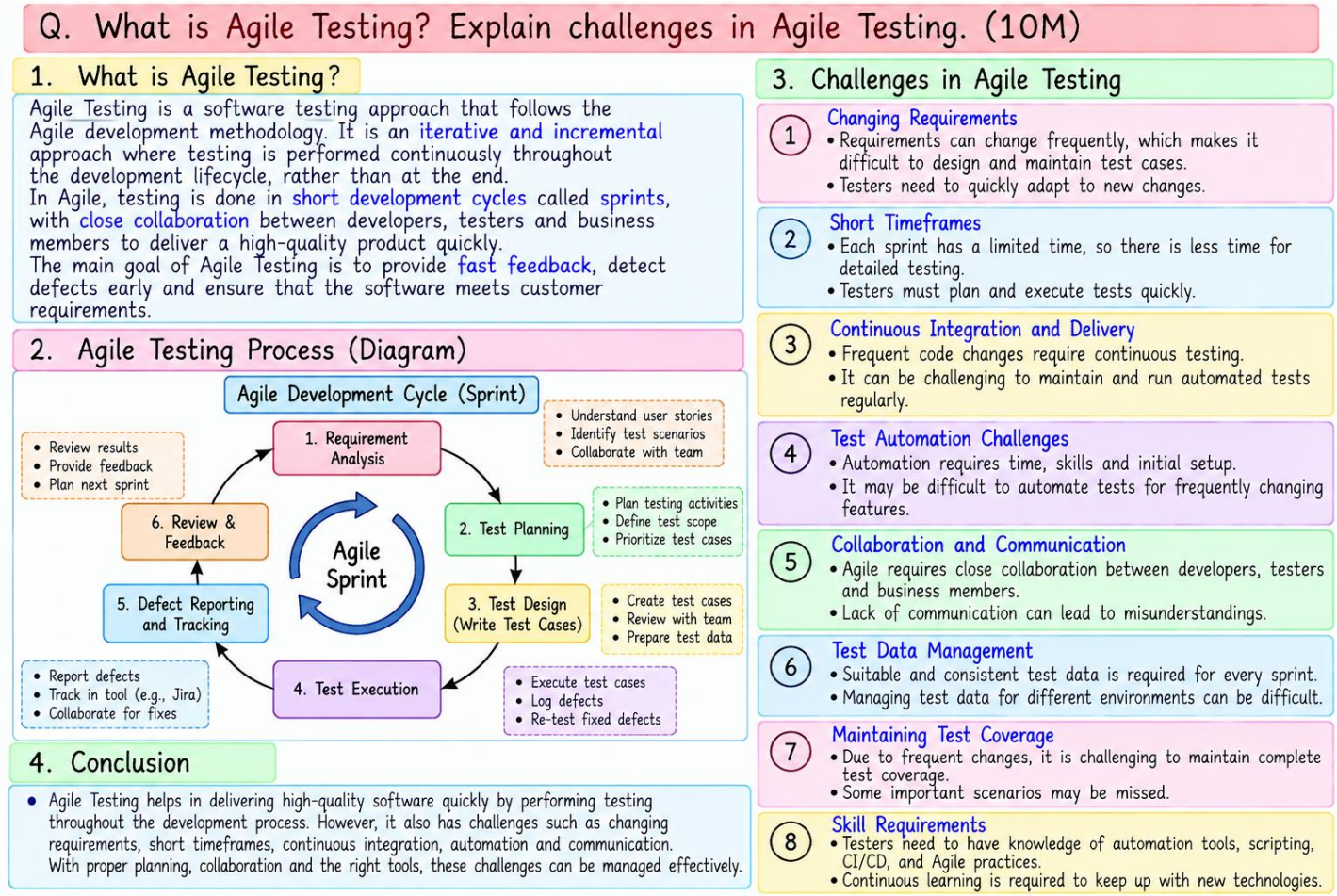
A tool with **good technical support, documentation and an active user community** is preferred.

**10. Stability and Reliability**

The tool should be **stable, reliable and regularly updated** to support new technologies and browsers.

Q7. What is Agile Testing? Explain challenges in Agile Testing. (8M)

ANS




**Definition of Agile Testing**

**Agile Testing** is a software testing approach that follows the **Agile development methodology**. In Agile, testing is performed **continuously throughout the development process**, rather than waiting until the development is completely finished.

Software is developed in small cycles called **Sprints**. Testers work closely with developers, business analysts and customers to find defects early and provide quick feedback.

**Main objective:**

To **detect defects early, provide fast feedback, and ensure that the software meets customer requirements**.

---

**2. Agile Testing Process**

The basic Agile testing process can be represented as:

**Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Review & Feedback → Next Sprint**

The process is repeated for every sprint.

**Explanation:**

1. **Requirement Analysis:**
   Tester understands user stories and identifies testing requirements.
2. **Test Planning:**
   Testing activities, priorities and required resources are planned.
3. **Test Design:**
   Test cases and test data are prepared according to requirements.
4. **Test Execution:**
   Test cases are executed and defects are identified.
5. **Defect Reporting:**
   Defects are reported and tracked using suitable tools.
6. **Review & Feedback:**
   Testing results are discussed and feedback is provided to improve the next sprint.

---

**3. Challenges in Agile Testing**

**1. Frequently Changing Requirements**

Agile requirements can change from one sprint to another. Therefore, testers need to **update test cases frequently**.

**2. Short Development Time**

Each sprint has a limited time. Testers have to **design, execute and report tests quickly**, which can create pressure.

**3. Continuous Integration**

Code is integrated frequently. Testing must also be performed continuously, making it difficult to maintain and execute all tests regularly.

**4. Test Automation Challenges**

Automation requires **time, technical skills and initial setup**. Frequently changing requirements can also make automation scripts difficult to maintain.

**5. Communication and Collaboration**

Agile requires close communication between **developers, testers, customers and business teams**. Poor communication may result in misunderstandings and missed requirements.

**6. Test Data Management**

Preparing suitable test data for every sprint and maintaining it across different environments can be difficult.

**7. Maintaining Test Coverage**

Due to frequent changes and short deadlines, it can be difficult to maintain **complete test coverage**. Some important test scenarios may be missed.

**8. Skill Requirements**

Agile testers need knowledge of **testing techniques, automation, scripting, CI/CD tools and Agile practices**. Continuous learning is often required.

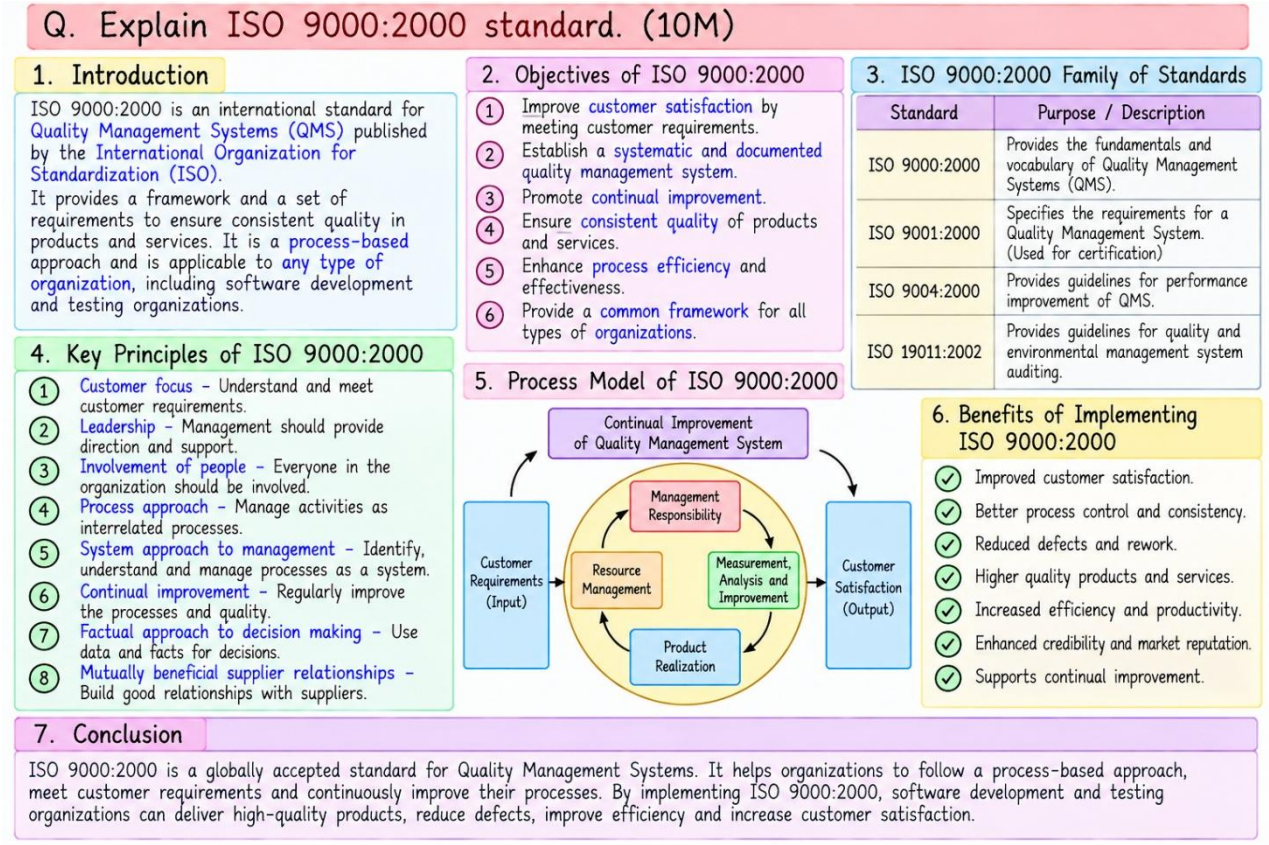
---

**4. Advantages of Agile Testing**

- Finds defects **early** in the development process.
- Provides **quick feedback** to developers.
- Improves software quality.
- Supports changing customer requirements.
- Encourages better communication between team members.
- Helps deliver working software **faster**.

Q8. Explain ISO 9000:2000 standard.

ANS




**Introduction**

**ISO 9000:2000** is an international standard related to **Quality Management Systems (QMS)**. It was developed by the **International Organization for Standardization (ISO)**.

It provides basic **principles, concepts and terminology** for establishing and maintaining an effective quality management system.

ISO 9000:2000 follows a **process-based approach** and focuses on customer satisfaction, consistent quality and continuous improvement.

---

**2. Objectives of ISO 9000:2000**

The main objectives are:

1. **Improve customer satisfaction** by meeting customer requirements.
2. Establish a **systematic and documented quality management system**.
3. Maintain **consistent quality** of products and services.
4. Encourage **continuous improvement** of processes.

5. Improve **process efficiency and effectiveness**.
6. Reduce defects, errors and unnecessary rework.
7. Provide a common framework for managing quality.

---

**3. ISO 9000:2000 Family of Standards**

| Standard | Purpose |
|---|---|
| **ISO 9000:2000** | Fundamentals and vocabulary of Quality Management Systems. |
| **ISO 9001:2000** | Specifies requirements for a Quality Management System and was used for certification. |
| **ISO 9004:2000** | Provides guidelines for improving the performance and effectiveness of a QMS. |

**Note:** ISO 19011:2002 provides guidelines for auditing quality and environmental management systems, but it is not itself part of the ISO 9000 family.

---

**4. Eight Quality Management Principles**

ISO 9000:2000 is based on **eight important principles**:

**1. Customer Focus**

The organization should understand and satisfy **customer needs and expectations**.

**2. Leadership**

Management should provide proper **direction, objectives and support** for quality improvement.

**3. Involvement of People**

Employees at all levels should participate in achieving quality objectives.

**4. Process Approach**

Activities should be managed as **related processes** to achieve better results.

**5. System Approach to Management**

Related processes should be identified and managed as a **complete system**.

**6. Continual Improvement**

The organization should continuously improve its **products, services and processes**.

**7. Factual Approach to Decision Making**

Decisions should be based on **data, facts and analysis** rather than assumptions.

**8. Mutually Beneficial Supplier Relationships**

The organization and its suppliers should maintain good relationships for **better quality and performance**.

---

**5. Process Model of ISO 9000:2000**

The basic process can be understood as:

**Customer Requirements → Quality Management Process → Customer Satisfaction**

The organization uses:

- **Management responsibility**
- **Resource management**
- **Product/service realization**
- **Measurement, analysis and improvement**

These activities lead to **continual improvement** of the Quality Management System.

**6. Benefits of ISO 9000:2000**

- Improves **customer satisfaction**.
- Provides better **process control**.
- Reduces defects and rework.
- Improves product and service quality.
- Increases efficiency and productivity.
- Provides a systematic approach to quality management.

- Improves the organization's **credibility and reputation**.
- Encourages **continuous improvement**.

UT-1

Q1. Explain Software Testing Life Cycle (STLC) with all its phases.

ANS

**Software Testing Life Cycle (STLC) – Phases**

**Software Testing Life Cycle (STLC)** is a systematic process followed by the testing team to ensure that a software application meets the specified requirements and works correctly. It consists of different phases, and each phase has specific activities, deliverables, and entry/exit criteria.

**Diagram of STLC**

**Requirement Analysis → Test Planning → Test Case Development → Test Environment Setup → Test Execution → Test Cycle Closure**

---

**1. Requirement Analysis**

In this phase, the testing team studies and analyzes the **Software Requirement Specification (SRS)** and other requirement documents.

**Activities:**

- Understand functional and non-functional requirements.
- Identify testable requirements.
- Identify the types of testing required.
- Prepare the **Requirement Traceability Matrix (RTM)**.

- Discuss unclear requirements with developers, business analysts, or clients.

**Deliverables:**

- Requirement Traceability Matrix (RTM)
- List of testable requirements
- Automation feasibility report, if required

---

**2. Test Planning**

In this phase, the **Test Manager or Test Lead** prepares a detailed plan for the testing process.

**Activities:**

- Define the scope and objectives of testing.
- Select the testing approach and strategy.
- Estimate testing effort and cost.
- Allocate resources and responsibilities.
- Identify testing tools.
- Prepare the testing schedule.
- Identify risks and prepare mitigation plans.

**Deliverable:**

- **Test Plan Document**

---

**3. Test Case Development**

In this phase, the testing team designs and develops test cases based on the requirements.

**Activities:**

- Prepare test scenarios.
- Design test cases.
- Prepare test data.

- Review and update test cases.
- Create automated test scripts, if automation is used.

**Deliverables:**

- Test cases
- Test scripts
- Test data

---

**4. Test Environment Setup**

The required hardware, software, network, database, and testing tools are prepared in this phase.

**Activities:**

- Set up the required hardware and software.
- Install and configure the application.
- Configure databases and networks.
- Verify that the test environment is ready.
- Perform a smoke test to check the basic functionality.

**Deliverable:**

- **Test Environment Readiness Report**

---

**5. Test Execution**

In this phase, the developed test cases are executed on the application.

**Activities:**

- Execute test cases.
- Compare actual results with expected results.
- Mark test cases as **Pass, Fail, or Blocked**.
- Report defects using a defect tracking tool.

- Perform retesting after defects are fixed.
- Perform regression testing to ensure that changes have not affected existing functionality.

**Deliverables:**

- Test execution report
- Defect report
- Updated RTM

---

**6. Test Cycle Closure**

This is the final phase of STLC. The testing team evaluates the complete testing process and prepares the final report.

**Activities:**

- Check whether all planned test cases have been executed.
- Analyze the number of defects found and fixed.
- Prepare test metrics.
- Document lessons learned.
- Prepare the **Test Closure Report**.
- Archive test documents and results for future reference.

**Deliverables:**

- Test Closure Report
- Test Metrics
- Lessons Learned Document

---

**Conclusion**

Thus, **STLC provides a structured approach to software testing**. It starts with requirement analysis and ends with test cycle closure. Each phase helps in identifying

defects systematically and ensures that the software meets the specified requirements and quality standards.

**Short Exam Diagram**

# SOFTWARE TESTING LIFE CYCLE (STLC)

## 1 Requirement Analysis

- Study and analyze the requirement documents (SRS).
- Identify testable requirements.
- Identify types of testing required.
- Prepare Requirement Traceability Matrix (RTM).
- Clarify doubts with stakeholders.

## 2 Test Planning

- Define the scope and objectives of testing.
- Decide test strategy, approach and resources.
- Estimate effort, cost and schedule.
- Identify tools and risks, prepare mitigation.
- Prepare Test Plan document.

## 3 Test Case Development

- Prepare test scenarios based on requirements.
- Design and develop test cases.
- Prepare test data.
- Review and update test cases.
- Develop automation scripts (if applicable).

## 4 Test Environment Setup

- Set up hardware, software and network.
- Install and configure the application.
- Configure databases and tools.
- Verify test environment readiness.
- Perform smoke test.

## 5 Test Execution

- Execute test cases.
- Compare actual results with expected results.
- Mark test cases as Pass / Fail / Blocked.
- Report defects in defect tracking tool.
- Perform retesting and regression testing.

## 6 Test Cycle Closure

- Ensure all test cases are executed.
- Analyze defects and test metrics.
- Prepare Test Closure Report.
- Document lessons learned.
- Archive test documents for future reference.

Q2. Differentiate between Effective Software Testing and Exhaustive Software Testing.

ANS

**Differentiate Between Effective Software Testing and Exhaustive Software Testing**

Software testing is the process of evaluating software to identify defects and verify whether it meets specified requirements. **Effective testing** focuses on finding the maximum number of defects with optimum time and resources, whereas **exhaustive testing** attempts to test all possible inputs, conditions, and execution paths.

| Parameter | Effective Software Testing | Exhaustive Software Testing |
|---|---|---|
| **Meaning** | Testing selected and important test cases to find maximum defects. | Testing all possible inputs, conditions, and paths of the software. |
| **Objective** | To find maximum defects with minimum time and resources. | To ensure every possible combination and scenario is tested. |
| **Number of Test Cases** | Uses a limited and optimized set of test cases. | Requires a very large number of test cases. |
| **Time Required** | Requires comparatively less time. | Requires a huge amount of time. |
| **Cost** | More cost-effective. | Very expensive due to extensive testing. |
| **Resources Required** | Requires optimum testing resources. | Requires a large amount of human and system resources. |
| **Practicality** | Practical and commonly used in real-world projects. | Generally impractical for complex software systems. |
| **Testing Approach** | Uses techniques such as equivalence partitioning, boundary value analysis, and risk-based testing. | Attempts to test every possible input, combination, and execution path. |
| **Defect Detection** | Focuses on detecting the most important and likely defects efficiently. | Can potentially detect defects in all tested combinations. |

| Parameter | Effective Software Testing | Exhaustive Software Testing |
|---|---|---|
| **Example** | Testing important valid, invalid, boundary, and high-risk inputs of a login system. | Testing every possible username, password, browser, device, and input combination. |
| **Efficiency** | High efficiency because effort is focused on important areas. | Low efficiency because many combinations may require excessive effort. |
| **Real-world Usage** | Widely used in software development projects. | Rarely possible except for very small and simple systems. |

**Key Difference**

**Effective Software Testing** means selecting the **right test cases** to find the maximum number of defects using minimum time, cost, and resources.

**Exhaustive Software Testing** means testing **all possible combinations, inputs, and paths**, which is usually impossible or impractical for large software applications.

Q3. Explain Model for Software Testing with a neat diagram.

ANS

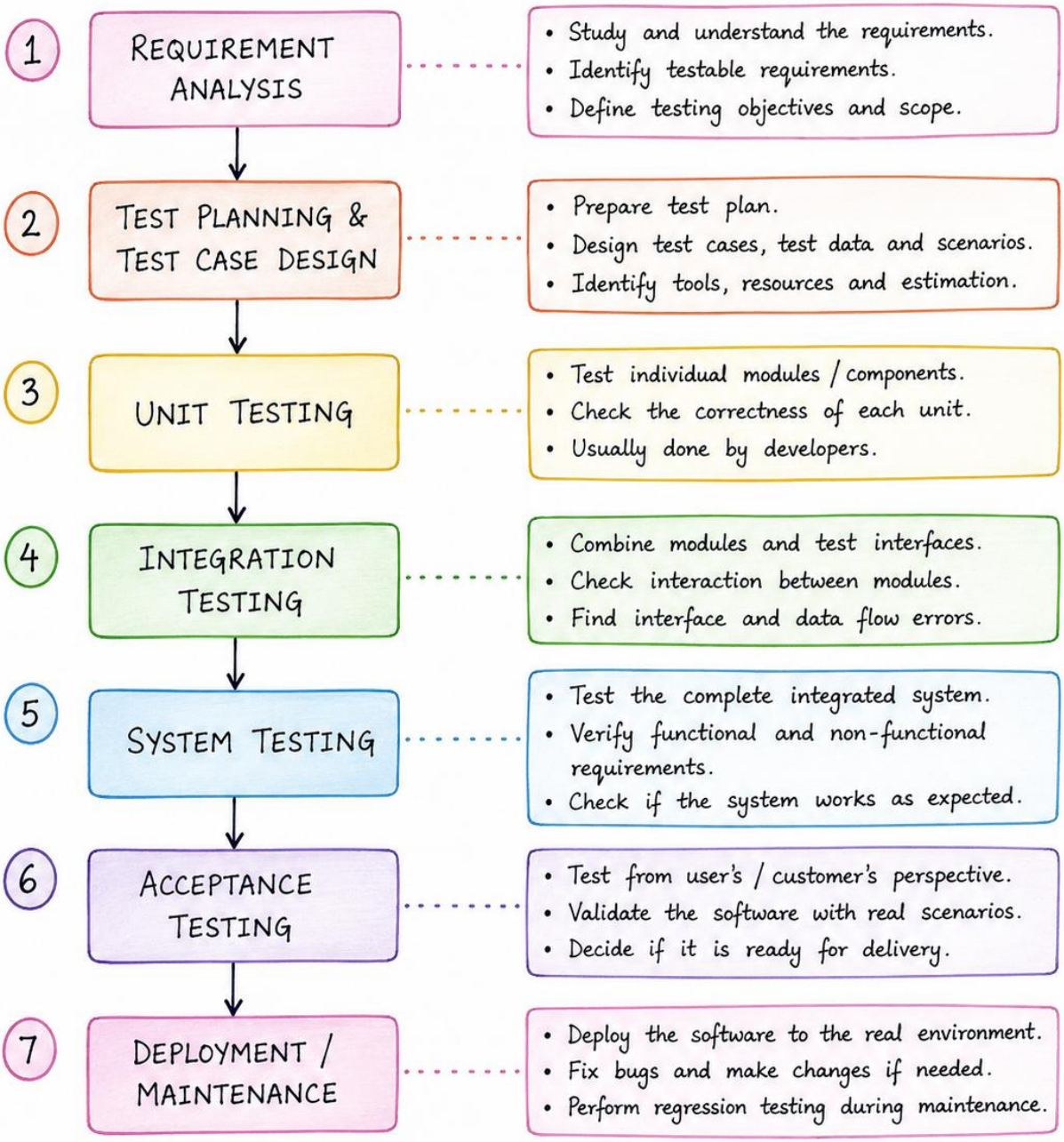
**Model for Software Testing — 10 Marks**

**Definition**

A **Software Testing Model** is a systematic approach used to verify and validate software at different stages of the Software Development Life Cycle (SDLC). It ensures that the software meets specified requirements and is free from defects.

**Neat Diagram**

# Model For Software Testing




**Purpose** : To ensure that the software is defect free, reliable and meets the customer requirements.

**Stages of Software Testing Model**

**1. Requirement Analysis**

- Testing team studies the functional and non-functional requirements.
- Testable requirements are identified.
- Testing objectives and scope are decided.

**2. Test Planning and Test Case Design**

- A test plan is prepared.
- Test cases, test data and testing techniques are designed.
- Required tools and resources are identified.

**3. Unit Testing**

- Individual modules or components are tested separately.
- It mainly verifies the correctness of individual functions or classes.
- Usually performed by developers.

**4. Integration Testing**

- Individual tested modules are combined and tested.
- It checks the interaction and communication between modules.
- Interface and data-flow errors are identified.

**5. System Testing**

- The complete integrated software system is tested.
- Functional and non-functional requirements are verified.
- It checks whether the complete system works as expected.

**6. Acceptance Testing**

- Testing is performed from the user's/customer's perspective.
- It determines whether the software is ready for delivery.
- User Acceptance Testing (UAT) is commonly performed at this stage.

**7. Deployment and Maintenance**

- After successful testing, the software is deployed.
- During maintenance, changes and fixes are again tested using regression testing.

**Advantages**

1. Provides a **systematic testing process**.
2. Helps detect defects at different stages.
3. Improves **software quality and reliability**.
4. Reduces the cost of fixing defects.
5. Ensures that software satisfies customer requirements.
6. Helps deliver a stable and dependable product.

**Conclusion**

The **Software Testing Model** provides a structured sequence of testing activities from requirement analysis to acceptance and maintenance. It helps identify defects early and ensures that the final software is **correct, reliable, and meets user requirements**.

Q4. Explain Software Failure Case Studies with suitable examples.

ANS

**Software Failure Case Studies – 10 Marks**

**Introduction**

A **software failure** occurs when a software system does not perform according to its specified requirements and causes incorrect results, financial loss, safety problems, or other serious consequences. Studying software failure cases helps developers understand the importance of **proper requirements, testing, design, coding, and maintenance**.

**1. Therac-25 Case Study**

**System:** Therac-25 radiation therapy machine
**Failure:** Software errors caused patients to receive extremely high doses of radiation.

**Cause:**

- Poor software design.

- Race conditions in the software.
- Insufficient testing of unusual situations.
- Heavy dependence on software without adequate hardware safety mechanisms.

**Impact:**

- Several patients received dangerous radiation doses.
- Some patients died or suffered serious injuries.

**Lesson:**
Safety-critical software must have proper testing, error handling, and independent safety mechanisms.

---

**2. Ariane 5 Rocket Failure**

**System:** Ariane 5 European rocket
**Failure:** The rocket exploded shortly after launch in 1996.

**Cause:**

- An old software component from Ariane 4 was reused.
- A numeric conversion caused an **integer overflow**.
- The error was not properly handled.

**Impact:**

- The rocket and its satellites were destroyed.
- Large financial loss occurred.

**Lesson:**
Software components should be thoroughly tested in their **new operating environment**, even if they worked correctly in another system.

---

**3. Mars Climate Orbiter Failure**

**System:** NASA Mars Climate Orbiter
**Failure:** The spacecraft was lost while entering Mars' atmosphere.

**Cause:**

- One software component used **English units** while another used **metric units**.
- The mismatch was not detected during system integration.

**Impact:**

- The spacecraft entered the wrong trajectory and was lost.
- NASA suffered a significant financial and scientific loss.

**Lesson:**
Clear specifications, standard units, interface checking, and integration testing are essential.

---

**4. Knight Capital Trading System Failure**

**System:** Automated stock-trading software
**Failure:** A software deployment caused the system to execute large numbers of incorrect trades.

**Cause:**

- An old piece of code was accidentally activated.
- Software deployment and testing procedures were inadequate.
- There was insufficient monitoring after deployment.

**Impact:**

- The company lost approximately **$440 million in about 45 minutes**.

**Lesson:**
Proper deployment procedures, regression testing, monitoring, and rollback mechanisms are essential.

---

**Common Causes of Software Failures**

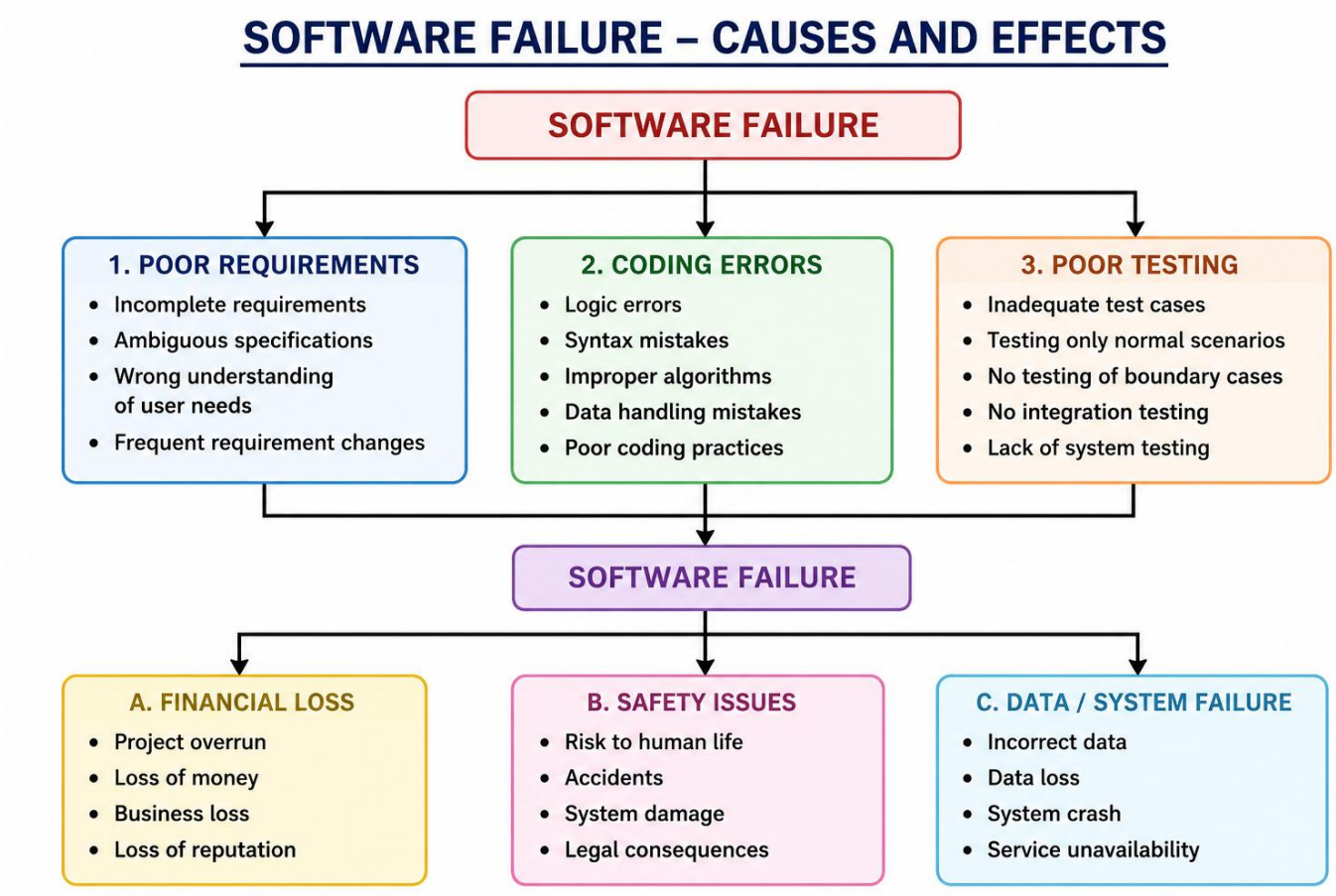
**Important Lessons from Case Studies**

1. Requirements must be **clear and complete**.
2. Software must be tested under **normal and abnormal conditions**.
3. Proper **unit, integration and system testing** is required.

4. Reused software must be tested in the new environment.
5. Interfaces and data formats must be clearly defined.
6. Proper deployment and rollback procedures should be followed.
7. Safety-critical systems require **fail-safe mechanisms**.
8. Continuous monitoring and maintenance help prevent failures.

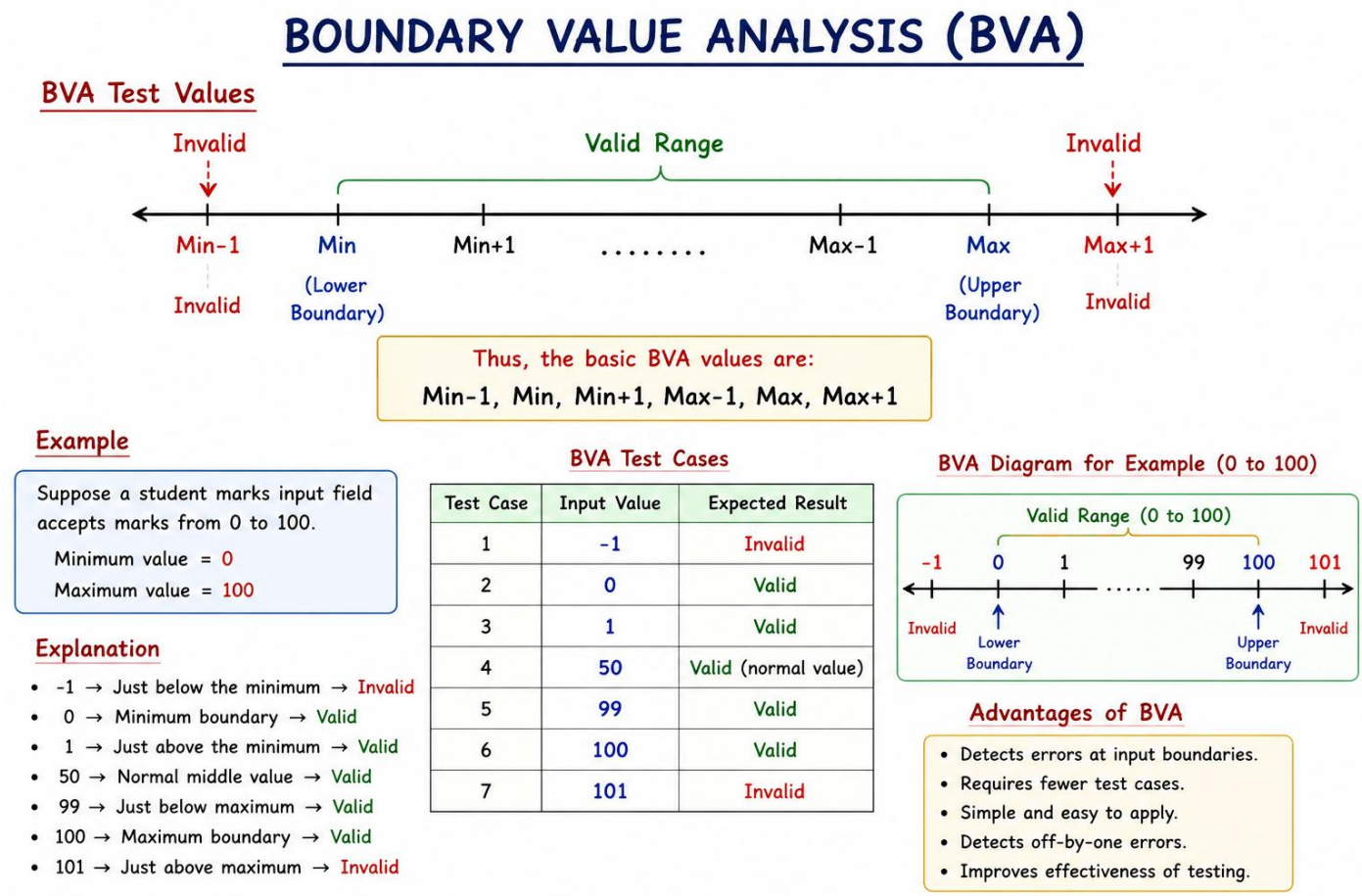
**Conclusion**

Software failure case studies show that even small errors in **requirements, design, coding, testing, or deployment** can cause major consequences. Therefore, following a systematic software development and testing process is essential for developing **reliable, safe, and high-quality software**.



Q5. Explain Boundary Value Analysis (BVA) with a suitable example.

ANS




**Boundary Value Analysis (BVA) – 10 Marks**

**Definition**

**Boundary Value Analysis (BVA)** is a **black-box software testing technique** in which test cases are designed using the values at the **boundaries of the input domain**.

The main idea is that software errors are more likely to occur at the **edges or boundaries** of valid input ranges than in the middle of the range.

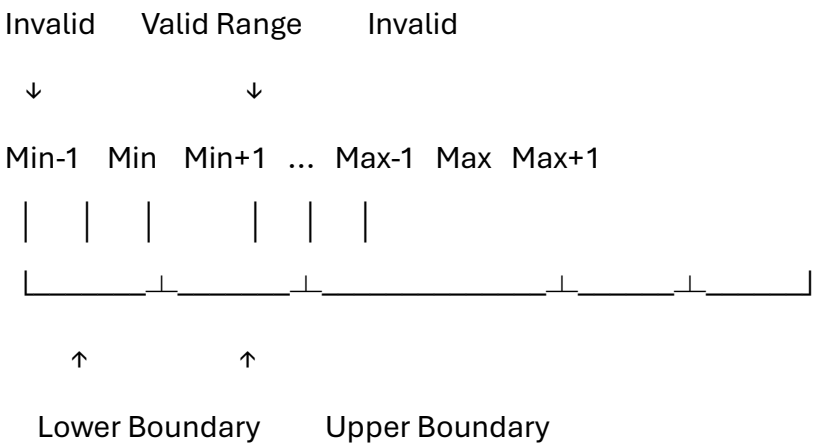
**Why BVA is Used?**

- To detect errors at input limits.
- To reduce the number of test cases.

- To test valid and invalid boundary conditions.
- To identify **off-by-one errors**.
- It is mainly used during functional/black-box testing.

**BVA Test Values**

For a variable having a valid range from **Minimum (Min) to Maximum (Max)**, the commonly selected values are:




Thus, the basic BVA values are:

**Min-1, Min, Min+1, Max-1, Max, Max+1**

---

**Suitable Example**

Suppose a **student marks input field** accepts marks from **0 to 100**.

- Minimum value = **0**
- Maximum value = **100**

Using BVA, we select:

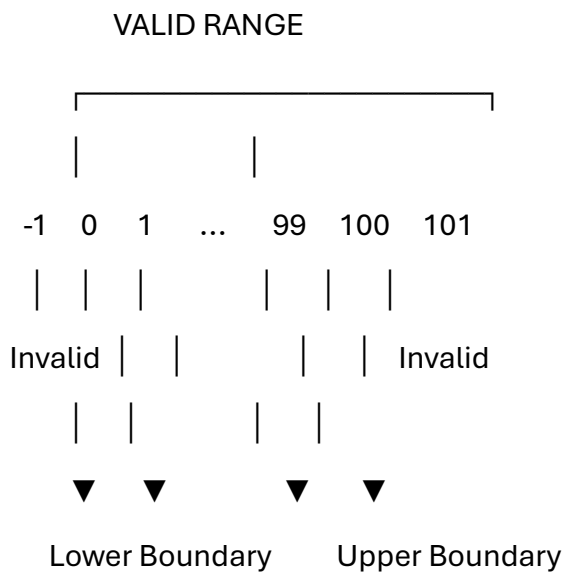
| Test Case | Input Value | Expected Result |
|---|---|---|
| 1 | -1 | Invalid |
| 2 | 0 | Valid |
| 3 | 1 | Valid |

| Test Case | Input Value | Expected Result |
|---|---|---|
| 4 | 50 | Valid (normal value) |
| 5 | 99 | Valid |
| 6 | 100 | Valid |
| 7 | 101 | Invalid |

**Explanation**

- **-1** → Just below the minimum → should be rejected.
- **0** → Minimum boundary → should be accepted.
- **1** → Just above the minimum → should be accepted.
- **50** → Normal middle value → should be accepted.
- **99** → Just below maximum → should be accepted.
- **100** → Maximum boundary → should be accepted.
- **101** → Just above maximum → should be rejected.

**BVA Diagram for Example**




**Advantages of BVA**

1. It detects errors occurring at input boundaries.

2. It requires fewer test cases compared with exhaustive testing.
3. It is simple and easy to apply.
4. It is effective for detecting **off-by-one errors**.
5. It improves the effectiveness of black-box testing.
6. It can be applied to numeric ranges, dates, sizes, lengths, etc.

**Limitations**

1. It mainly focuses on boundaries and may miss errors in the middle of the range.
2. It is less useful when there are no clearly defined input boundaries.
3. Multiple-input conditions may require additional test cases.
4. It does not examine the internal code structure.

**Conclusion**

**Boundary Value Analysis** is an important black-box testing technique that tests values **at, just below, and just above the boundaries** of an input range. For a range of **0–100**, values such as **-1, 0, 1, 99, 100 and 101** help identify boundary-related defects effectively.

Q6. Explain Cause-Effect Graphing Based Testing with an example

ANS

**Cause-Effect Graphing Based Testing – 10 Marks**

**Definition**

**Cause-Effect Graphing Based Testing** is a **black-box testing technique** used to identify the relationship between **input conditions (causes)** and **output conditions (effects)** of a software system.

It represents the logical relationship between causes and effects using a **graph**, and then converts the graph into a **decision table** to generate effective test cases.

**Basic Concept**

- **Cause** → An input condition or event.

- **Effect** → An output condition or action.
- **Logical operators** such as **AND, OR, and NOT** are used to connect causes and effects.
- The graph is converted into a **decision table**, from which test cases are prepared.

**Steps of Cause-Effect Graphing**

1. **Identify the causes** from the software requirements.
2. **Identify the effects** expected from the system.
3. Establish the logical relationships between causes and effects.
4. Draw the **cause-effect graph** using logical conditions.
5. Convert the graph into a **decision table**.
6. Generate test cases from the decision table.
7. Execute the test cases and verify the results.

---

**Suitable Example**

Consider a **login system** with the following requirements:

- The user must enter a **valid username**.
- The user must enter a **valid password**.
- If both are valid, **login is successful**.
- If either username or password is invalid, **login fails**.

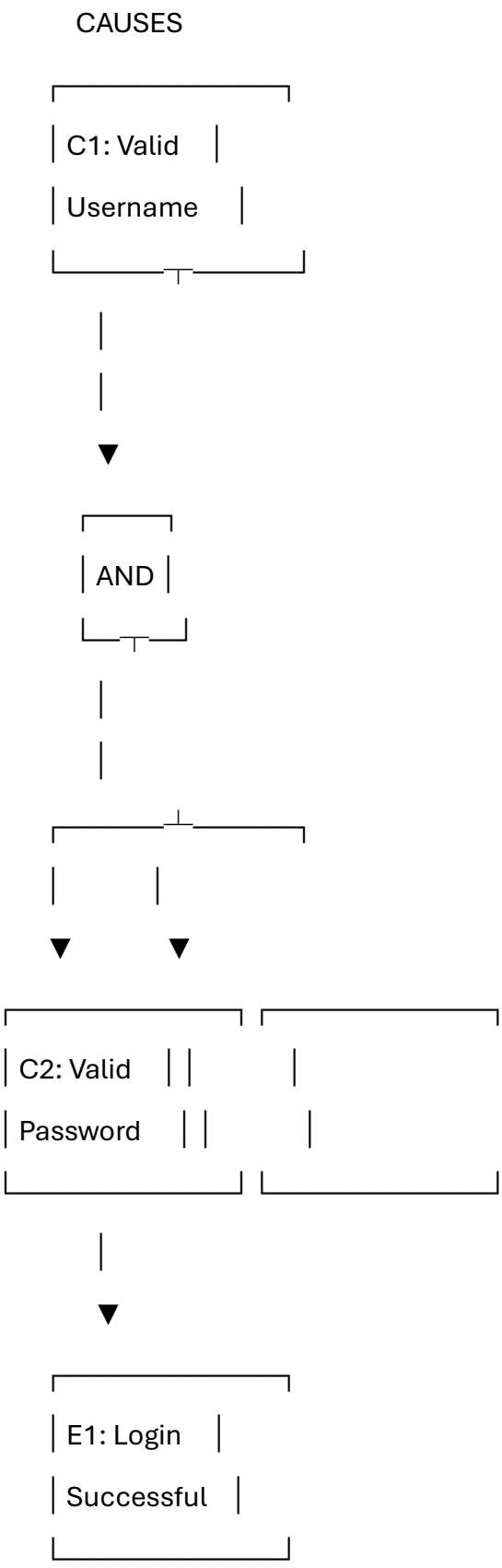
**Causes**

- **C1:** Username is valid.
- **C2:** Password is valid.

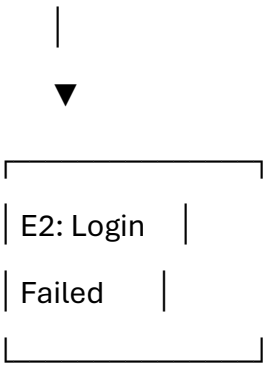
**Effects**

- **E1:** Login successful.
- **E2:** Login failed.

**Cause-Effect Graph**


CAUSES
C1: Valid Username
AND
C2: Valid Password
E1: Login Successful

If C1 AND C2 = FALSE




**Decision Table**

| Test Case | Valid Username (C1) | Valid Password (C2) | Expected Result |
|---|---|---|---|
| TC1 | Yes | Yes | Login Successful |
| TC2 | Yes | No | Login Failed |
| TC3 | No | Yes | Login Failed |
| TC4 | No | No | Login Failed |

**Explanation**

- **TC1:** Both username and password are valid → **Login Successful**.
- **TC2:** Username is valid but password is invalid → **Login Failed**.
- **TC3:** Username is invalid but password is valid → **Login Failed**.
- **TC4:** Both are invalid → **Login Failed**.

**Advantages**

1. Provides a **systematic way of designing test cases**.
2. Tests combinations of input conditions.
3. Helps identify missing or ambiguous requirements.
4. Reduces unnecessary test cases.
5. Useful for testing systems containing complex logical conditions.

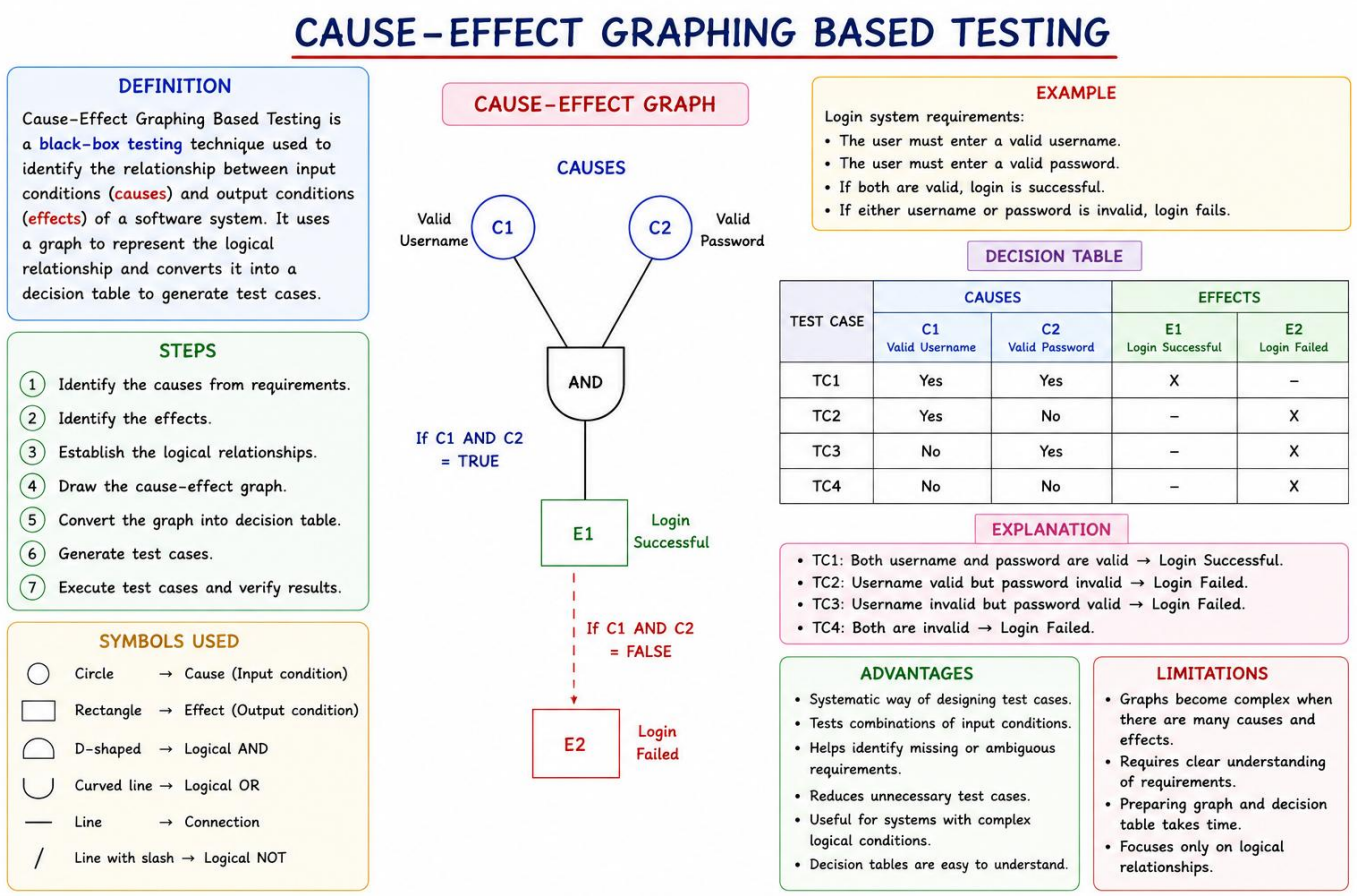
6. Decision tables make the test cases easy to understand.

**Limitations**

1. Graphs can become complex when there are many causes and effects.
2. Requires a clear understanding of software requirements.
3. Preparing the graph and decision table can be time-consuming.
4. It mainly focuses on logical relationships between inputs and outputs.

**Conclusion**

**Cause-Effect Graphing Based Testing** is an effective black-box testing technique that establishes a relationship between **causes (inputs)** and **effects (outputs)**. The cause-effect graph is converted into a **decision table**, which helps generate systematic and comprehensive test cases.



Q7. Explain Mutation Testing.

ANS

**Mutation Testing – 10 Marks**

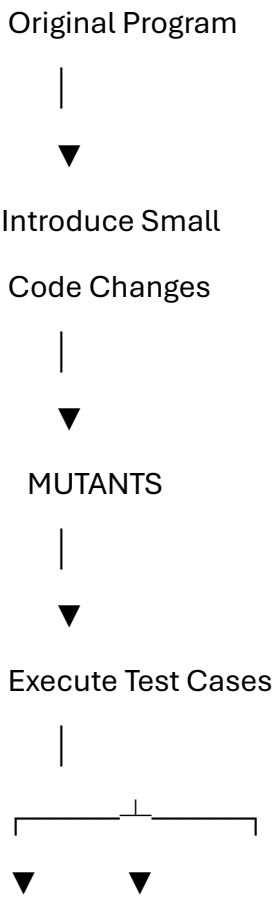
**Definition**

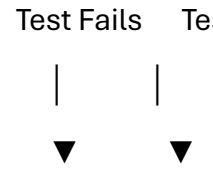
**Mutation Testing** is a software testing technique used to evaluate the **effectiveness of existing test cases**. In this technique, small changes are intentionally introduced into the program source code. These modified versions are called **mutants**.

The test cases are then executed on the mutants. If the test cases detect the introduced error, the mutant is said to be **killed**. If the mutant produces the same result as the original program, it **survives**.

---

**Basic Concept**






**Example**

Consider the following program:

Original Code:

```
if (age >= 18)
   print("Eligible");
else
   print("Not Eligible");
```

A mutation can be created by changing the operator:

Mutant Code:

```
if (age > 18)
   print("Eligible");
else
   print("Not Eligible");
```

Here, >= is changed to >.

Now consider the test case:

**Input: age = 18**

| Program | Condition | Expected Result |
|---|---|---|
| Original | 18 >= 18 | Eligible |
| Mutant | 18 > 18 | Not Eligible |

The test case detects the difference, so the **mutant is killed**.

---

**Types of Mutations**

**1. Arithmetic Operator Mutation**

Arithmetic operators are changed.

Example:

a + b → a - b

a * b → a / b

**2. Relational Operator Mutation**

Relational operators are changed.

Example:

a > b → a >= b

a == b → a != b

**3. Logical Operator Mutation**

Logical operators are modified.

Example:

A AND B → A OR B

**4. Conditional Mutation**

Conditions are changed or removed.

Example:

if (A && B) → if (A || B)

**5. Constant Mutation**

Constants are replaced with other values.

Example:

if (x > 10) → if (x > 20)

---

**Mutation Testing Process**

1. **Select the original program.**

2. **Create mutants** by making small changes to the program.
3. **Run existing test cases** on each mutant.
4. If a test case detects the mutation, the mutant is **killed**.
5. If no test case detects it, the mutant **survives**.
6. Analyze surviving mutants and improve the test cases.
7. Repeat the process until the test suite becomes sufficiently effective.

**Mutation Score**

The effectiveness of test cases can be measured using the **Mutation Score**:

**Example:**

If 90 mutants are created and 81 are killed:

Therefore, the test suite has a **90% mutation score**.

---

**Advantages**

1. Measures the **quality and effectiveness of test cases**.
2. Helps identify weak or missing test cases.
3. Detects faults that normal testing may miss.
4. Improves the overall quality of the test suite.
5. Provides a quantitative **mutation score**.
6. Useful for testing critical software.

**Limitations**

1. Mutation testing can be **time-consuming**.
2. A large number of mutants may require significant computational resources.
3. Creating and executing mutants can be expensive.
4. Some mutants may be equivalent to the original program and cannot be killed by any test case.
5. Automated tools are generally required for large projects.

**Conclusion**

**Mutation Testing** is an effective technique for checking how well a test suite can detect faults. By creating small changes (**mutants**) in the program and checking whether the test cases **kill or allow them to survive**, developers can identify weaknesses in testing and improve software quality.

Q8. Explain Progressive Testing and Regressive Testing.

ANS

**Progressive Testing and Regressive Testing**

**1. Progressive Testing**

**Progressive Testing** is a testing approach in which **new features or enhancements are tested as they are added to the software**. The purpose is to check whether the newly developed functionality works correctly.

**Example:**
Suppose an online shopping application initially has **Login** and **Product Search** features. Later, a **Payment** feature is added. Progressive testing checks whether the new Payment feature works correctly with the existing system.

**Steps:**

1. Develop a new feature.
2. Integrate it with the existing system.
3. Test the newly added feature.
4. Fix defects found during testing.
5. Continue testing as new features are added.

---

**2. Regressive Testing**

**Regressive Testing (Regression Testing)** is performed to ensure that **changes or modifications made to the software have not affected existing functionalities**.

**Example:**
If the Payment module is modified in an online shopping application, regression testing checks whether **Login, Product Search, Cart, and other existing features** still work correctly.

**Steps:**

1. Modify or update the software.
2. Identify affected existing functions.
3. Re-run previously executed test cases.
4. Check whether existing functionality still works.
5. Fix any newly introduced defects.

**Difference between Progressive and Regressive Testing**

| **Progressive Testing** | **Regressive Testing** |
|---|---|
| Tests newly added features. | Tests existing features after changes. |
| Focuses on new functionality. | Focuses on detecting side effects of changes. |
| Performed when new features are developed. | Performed after modifications, bug fixes or updates. |
| Ensures new functionality works correctly. | Ensures old functionality continues to work correctly. |
| Example: Testing a newly added Payment module. | Example: Checking Login and Cart after Payment module changes. |

**In short:**
**Progressive Testing → "Does the new feature work?"**
**Regressive Testing → "Did the change break anything that was already working?"**